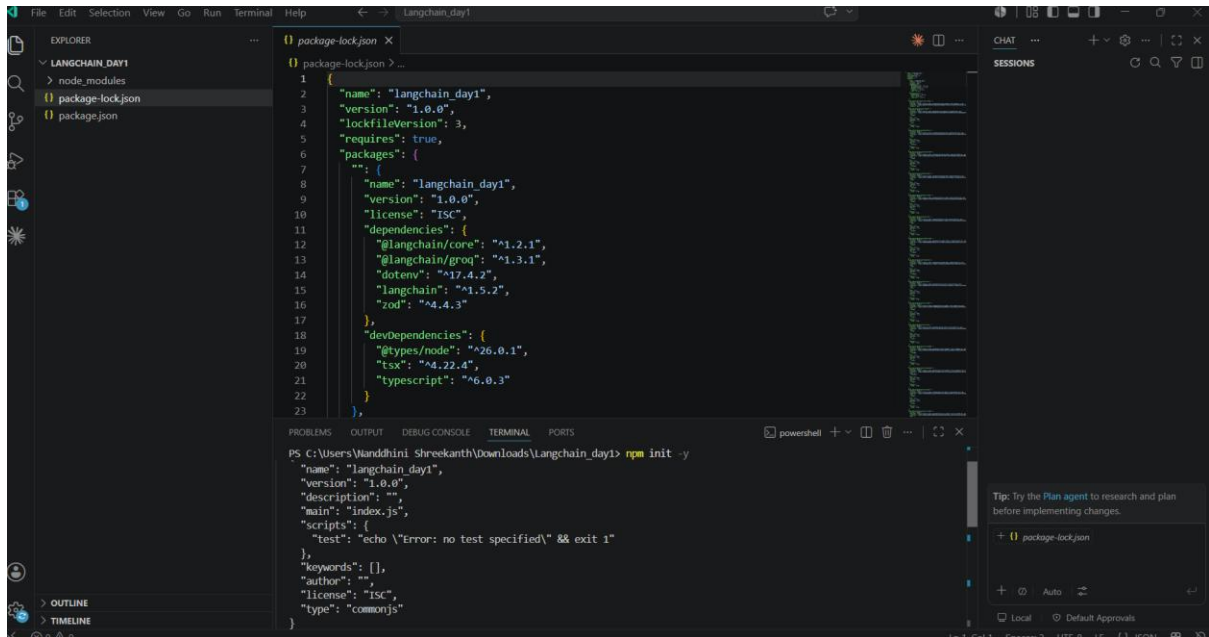


1.LangChain >> Model

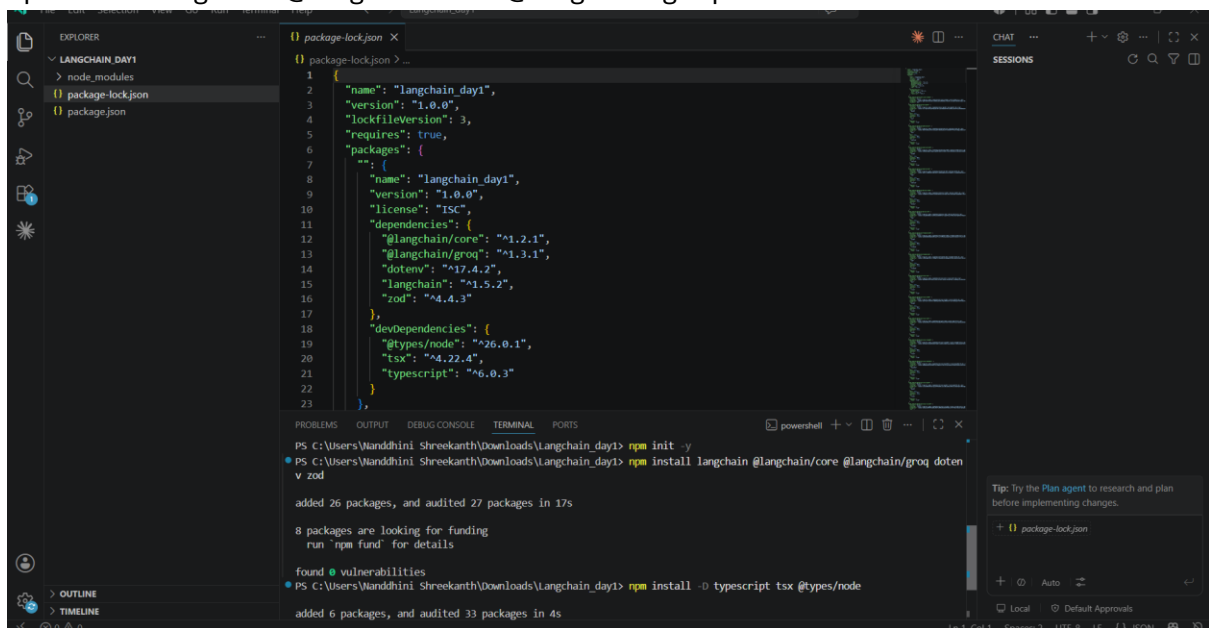
npm init -y



```
1 {
2   "name": "langchain_day1",
3   "version": "1.0.0",
4   "lockfileVersion": 3,
5   "requires": true,
6   "packages": {
7     "": {
8       "name": "langchain_day1",
9       "version": "1.0.0",
10      "license": "ISC",
11      "dependencies": {
12        "@langchain/core": "^1.2.1",
13        "@langchain/groq": "^1.3.1",
14        "dotenv": "^17.4.2",
15        "langchain": "^1.5.2",
16        "zod": "^4.4.3"
17      },
18      "devDependencies": {
19        "@types/node": "^26.0.1",
20        "tsx": "^4.22.4",
21        "typescript": "^6.0.3"
22      }
23    }
24  }
25 }
```

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm init -y
"name": "langchain_day1",
"version": "1.0.0",
"description": "",
"main": "index.js",
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "commonjs"
```

npm install langchain @langchain/core @langchain/groq dotenv zod



```
1 {
2   "name": "langchain_day1",
3   "version": "1.0.0",
4   "lockfileVersion": 3,
5   "requires": true,
6   "packages": {
7     "": {
8       "name": "langchain_day1",
9       "version": "1.0.0",
10      "license": "ISC",
11      "dependencies": {
12        "@langchain/core": "^1.2.1",
13        "@langchain/groq": "^1.3.1",
14        "dotenv": "^17.4.2",
15        "langchain": "^1.5.2",
16        "zod": "^4.4.3"
17      },
18      "devDependencies": {
19        "@types/node": "^26.0.1",
20        "tsx": "^4.22.4",
21        "typescript": "^6.0.3"
22      }
23    }
24  }
25 }
```

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install -y
added 26 packages, and audited 27 packages in 17s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install -y typescript tsx @types/node
added 6 packages, and audited 33 packages in 4s
```

npm install -D typescript tsx @types/node

The screenshot shows the VS Code interface with the Explorer, Source Control, and Run and Debug views. The Explorer view shows the file structure of the project, including the package-lock.json and package.json files. The Source Control view shows the changes made to the package-lock.json file. The Run and Debug view shows the terminal output of the command `npm install -D typescript tsx @types/node`. The terminal output shows that the command was successful and that the packages were installed.

```
package-lock.json
{
  "name": "langchain_day1",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "langchain_day1",
      "version": "1.0.0",
      "license": "ISC",
      "dependencies": {
        "@langchain/core": "^1.2.1",
        "@langchain/groq": "^1.3.1",
        "dotenv": "^17.4.2",
        "langchain": "^1.5.2",
        "zod": "^4.4.3"
      },
      "devDependencies": {
        "@types/node": "^26.0.1",
        "tsx": "^4.22.4",
        "typescript": "^6.0.3"
      }
    }
  }
}
```

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install langchain @langchain/core @langchain/groq dotenv zod
added 6 packages, and audited 33 packages in 4s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install -D typescript tsx @types/node
added 6 packages, and audited 33 packages in 4s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

npx tsc --init

The screenshot shows the VS Code interface with the Explorer, Source Control, and Run and Debug views. The Explorer view shows the file structure of the project, including the package-lock.json, package.json, and tsconfig.json files. The Source Control view shows the changes made to the package-lock.json file. The Run and Debug view shows the terminal output of the command `npx tsc --init`. The terminal output shows that the command was successful and that a new tsconfig.json file was created.

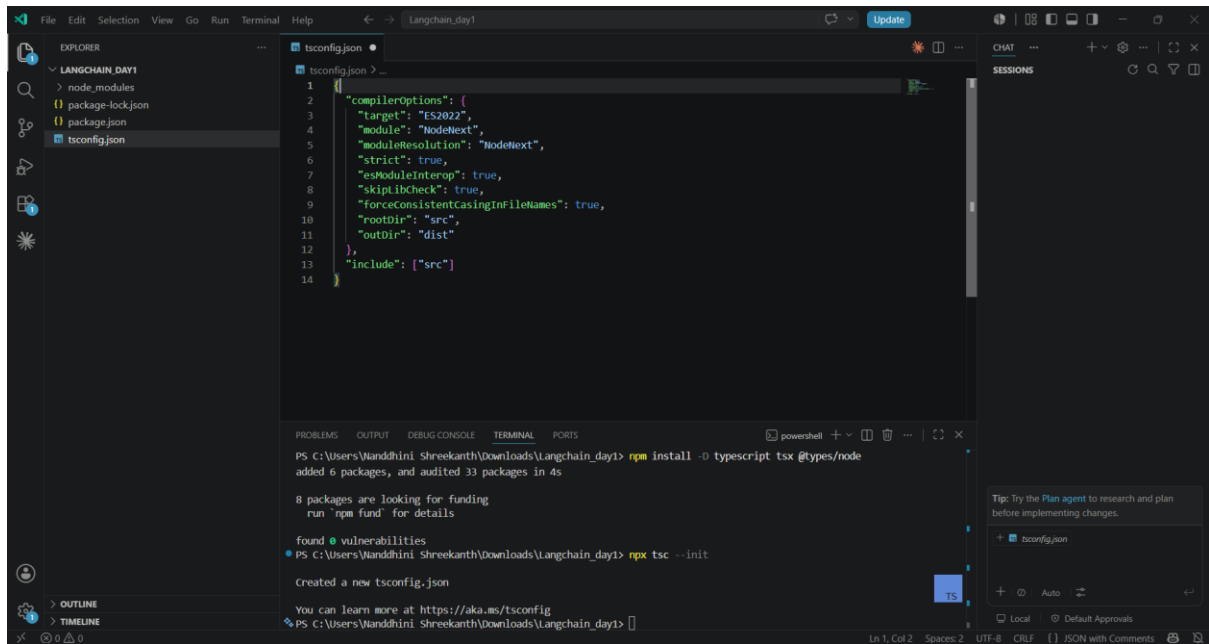
```
package-lock.json
{
  "name": "langchain_day1",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "langchain_day1",
      "version": "1.0.0",
      "license": "ISC",
      "dependencies": {
        "@langchain/core": "^1.2.1",
        "@langchain/groq": "^1.3.1",
        "dotenv": "^17.4.2",
        "langchain": "^1.5.2",
        "zod": "^4.4.3"
      },
      "devDependencies": {
        "@types/node": "^26.0.1",
        "tsx": "^4.22.4",
        "typescript": "^6.0.3"
      }
    }
  }
}
```

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npx tsc --init
Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig

PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

Replaced the below code in tsconfig.json



The screenshot shows the Visual Studio Code interface with the `tsconfig.json` file open in the editor. The file contains the following configuration:

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "NodeNext",
5     "moduleResolution": "NodeNext",
6     "strict": true,
7     "esModuleInterop": true,
8     "skipLibCheck": true,
9     "forceConsistentCasingInFileNames": true,
10    "rootDir": "src",
11    "outDir": "dist"
12  },
13  "include": ["src"]
14 }
```

The Explorer sidebar on the left shows the project structure with `src` highlighted. The terminal at the bottom shows the following commands and output:

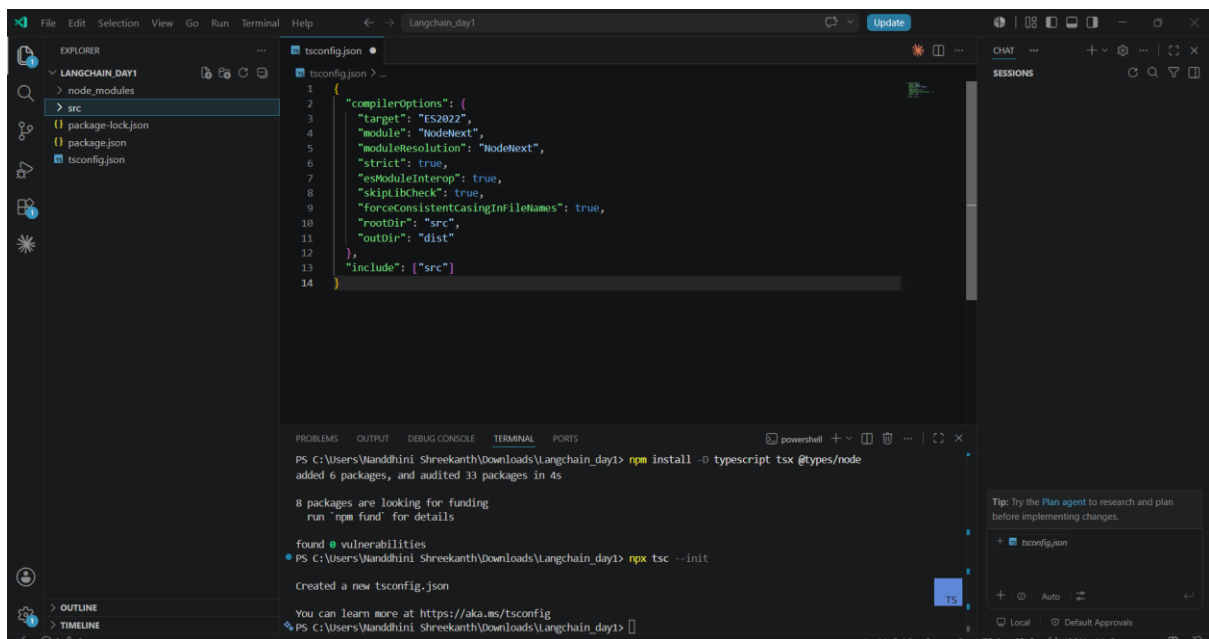
```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install -D typescript tsx @types/node
added 6 packages, and audited 33 packages in 4s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npx tsc --init
Created a new tsconfig.json

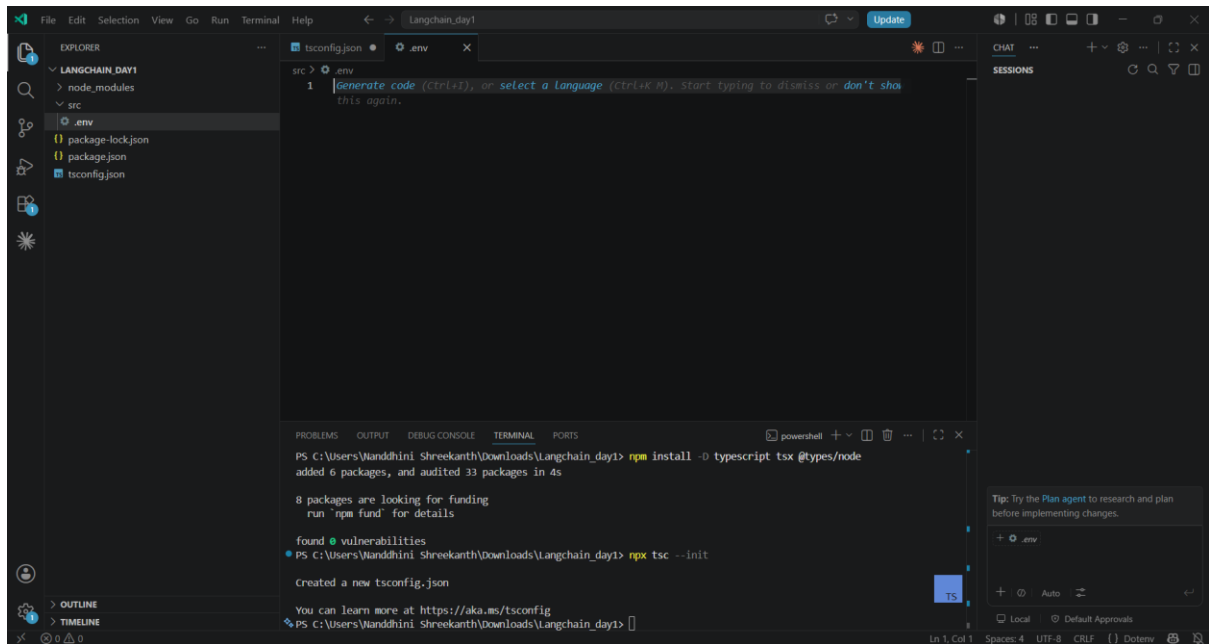
You can learn more at https://aka.ms/tsconfig
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

Create a new folder in the project and name it as src

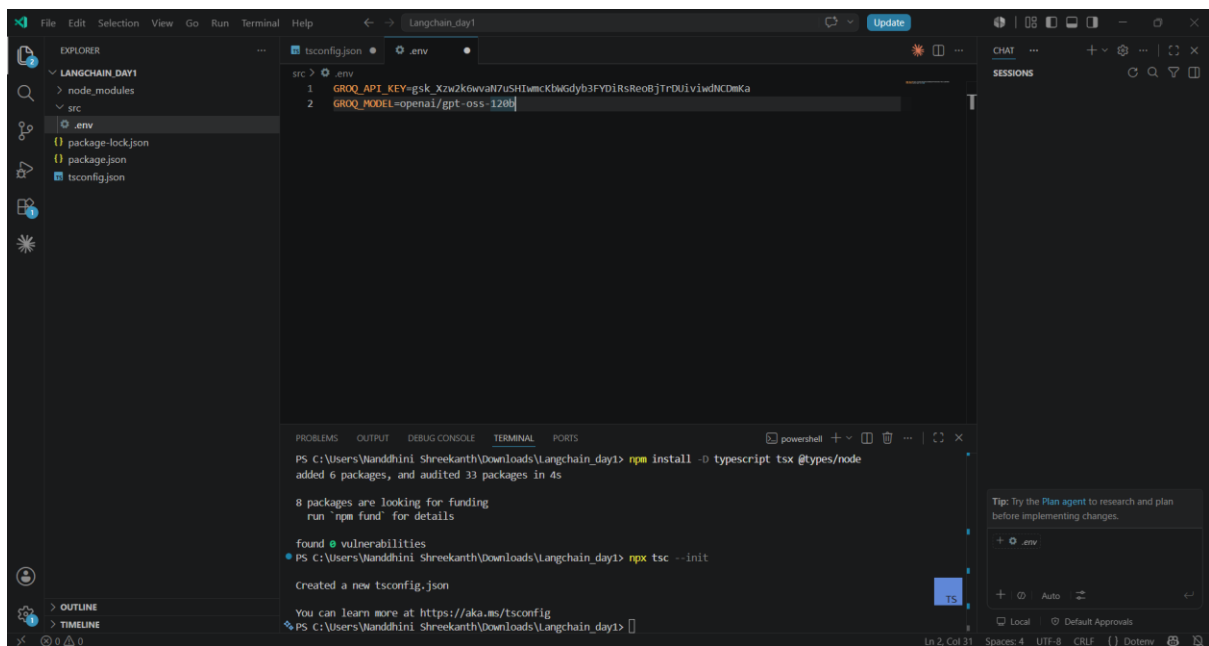


The screenshot shows the Visual Studio Code interface after creating the `src` folder. The Explorer sidebar on the left shows the project structure with `src` highlighted. The `tsconfig.json` file is still open in the editor, showing the same configuration as before. The terminal at the bottom shows the same commands and output as the previous screenshot.

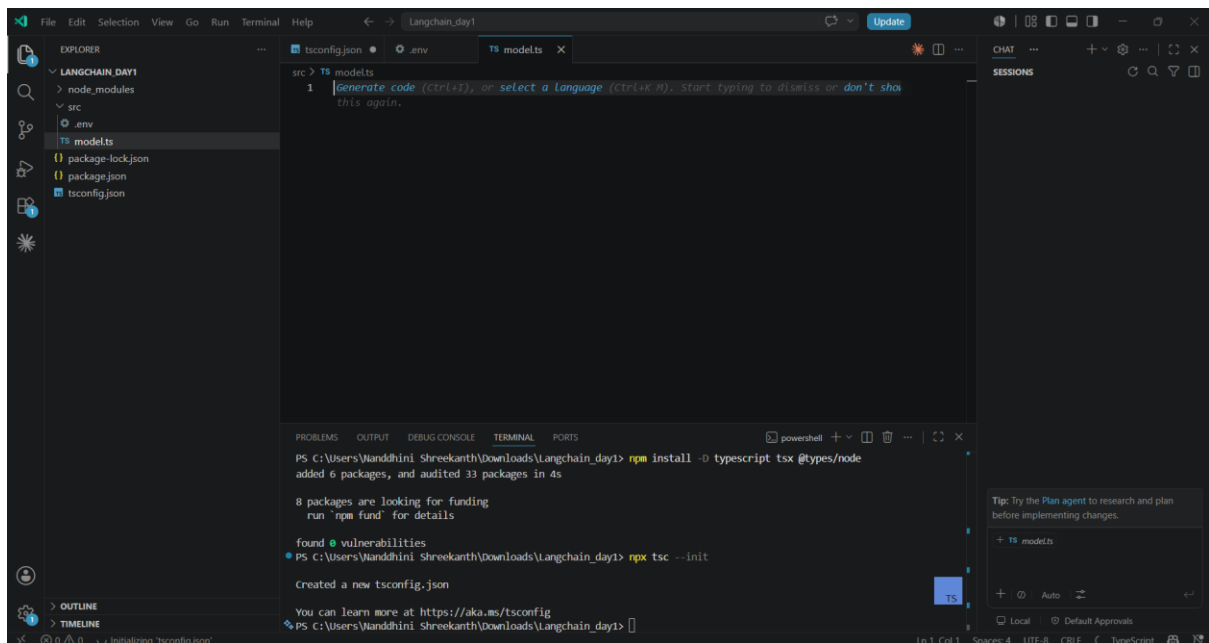
Create .env file in the Folder



Add GROQ_API_KEY and GROQ_MODEL in .env file



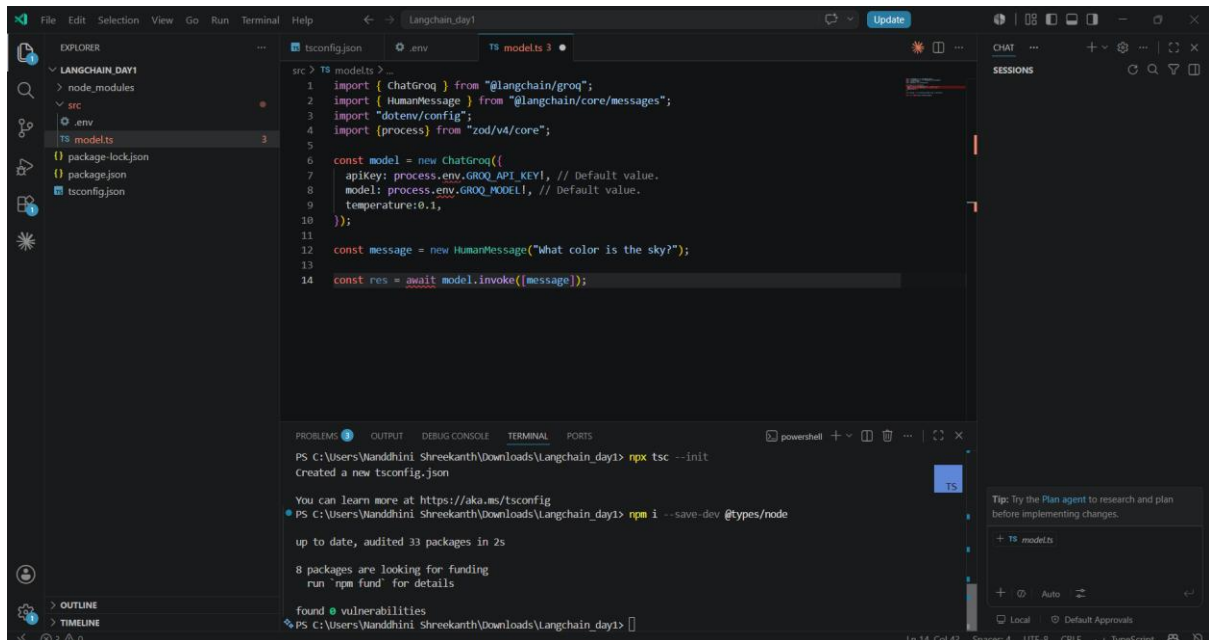
Create a file by right clicking the src folder and rename it as model.ts



Copy the below code in Model.ts

```
import { ChatGroq } from "@langchain/groq";  
  
import { HumanMessage } from "@langchain/core/messages";  
  
const model = new ChatGroq({  
  apiKey: process.env.GROQ_API_KEY, // Default value.  
  model: "llama-3.3-70b-versatile",  
});  
  
const message = new HumanMessage("What color is the sky?");  
  
const res = await model.invoke([message]);
```

In model.ts, add the ! mark in line number 6 and 7 and In line 7 and add the temperature as 0.1 and add the import “dotenv/config”; in line number 3



```
1 import { ChatGrok } from "@langchain/grok";
2 import { HumanMessage } from "@langchain/core/messages";
3 import "dotenv/config";
4 import { process } from "zod/v4/core";
5
6 const model = new ChatGrok({
7   apiKey: process.env.GROQ_API_KEY!, // Default value.
8   model: process.env.GROQ_MODEL!, // Default value.
9   temperature: 0.1,
10 });
11
12 const message = new HumanMessage("What color is the sky?");
13
14 const res = await model.invoke([message]);
```

PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npx tsc --init
Created a new tsconfig.json

You can learn more at <https://aka.ms/tsconfig>

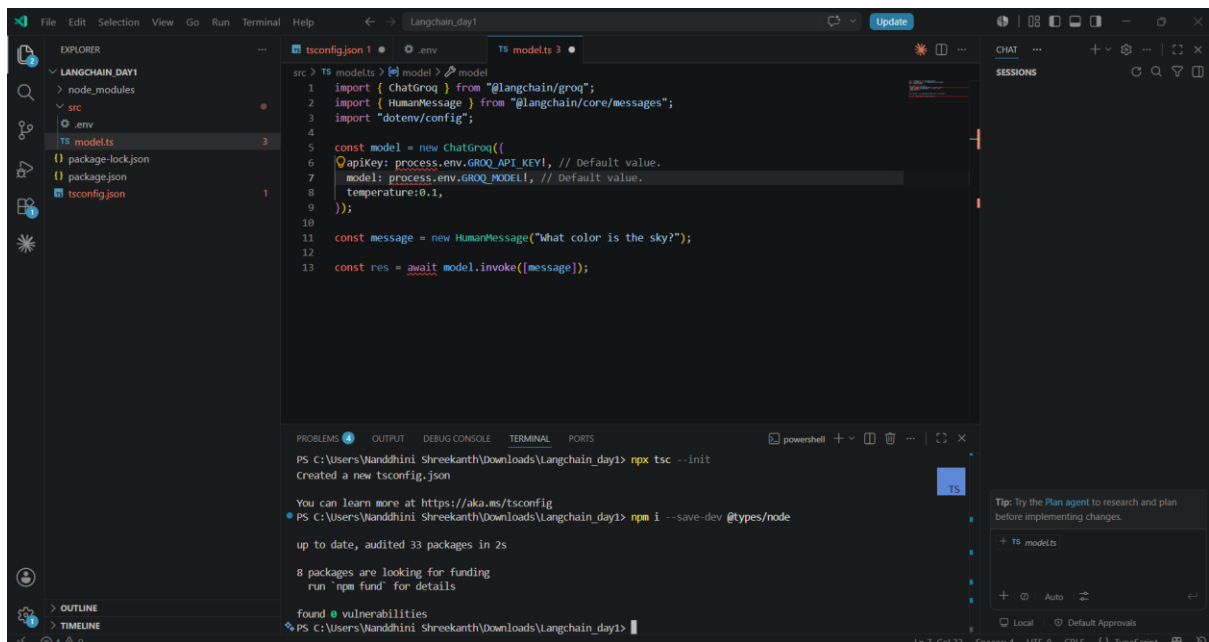
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm i --save-dev @types/node

up to date, audited 33 packages in 2s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

Run npm i --save-dev @types/node in terminal



```
1 import { ChatGrok } from "@langchain/grok";
2 import { HumanMessage } from "@langchain/core/messages";
3 import "dotenv/config";
4
5 const model = new ChatGrok({
6   apiKey: process.env.GROQ_API_KEY!, // Default value.
7   model: process.env.GROQ_MODEL!, // Default value.
8   temperature: 0.1,
9 });
10
11 const message = new HumanMessage("What color is the sky?");
12
13 const res = await model.invoke([message]);
```

PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npx tsc --init
Created a new tsconfig.json

You can learn more at <https://aka.ms/tsconfig>

PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm i --save-dev @types/node

up to date, audited 33 packages in 2s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

Add "types": ["node"], in tsconfig.json

The screenshot shows the Visual Studio Code editor with the `tsconfig.json` file open. The file content is as follows:

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "NodeNext",
5     "moduleResolution": "NodeNext",
6     "strict": true,
7     "esModuleInterop": true,
8     "types": ["node"],
9     "skipLibCheck": true,
10    "forceConsistentCasingInFileNames": true,
11    "rootDir": "src",
12    "outDir": "dist"
13  },
14  "include": ["src"]
15 }
```

The Explorer sidebar on the left shows the project structure for `LANGCHAIN_DAY1`, including `node_modules`, `src`, `.env`, `ts`, `models`, `package-lock.json`, `package.json`, and `tsconfig.json`. The terminal at the bottom shows the following commands and output:

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm tsc --init
Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm i --save-dev @types/node
up to date, audited 33 packages in 2s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

Change the line number 12 type as Module in package.json

The screenshot shows the Visual Studio Code editor with the `package.json` file open. The file content is as follows:

```
1 {
2   "name": "langchain_day1",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "type": "module",
13  "dependencies": {
14    "@langchain/core": "^1.2.1",
15    "@langchain/groq": "^1.3.1",
16    "dotenv": "^17.4.2",
17    "langchain": "^1.5.2",
18    "zod": "^4.4.3"
19  },
20  "devDependencies": {
21    "@types/node": "^26.0.1",
22    "tsx": "^4.22.4"
23  }
24 }
```

The Explorer sidebar on the left shows the project structure for `LANGCHAIN_DAY1`, including `node_modules`, `src`, `.env`, `ts`, `models`, `package-lock.json`, `package.json`, and `tsconfig.json`. The terminal at the bottom shows the following commands and output:

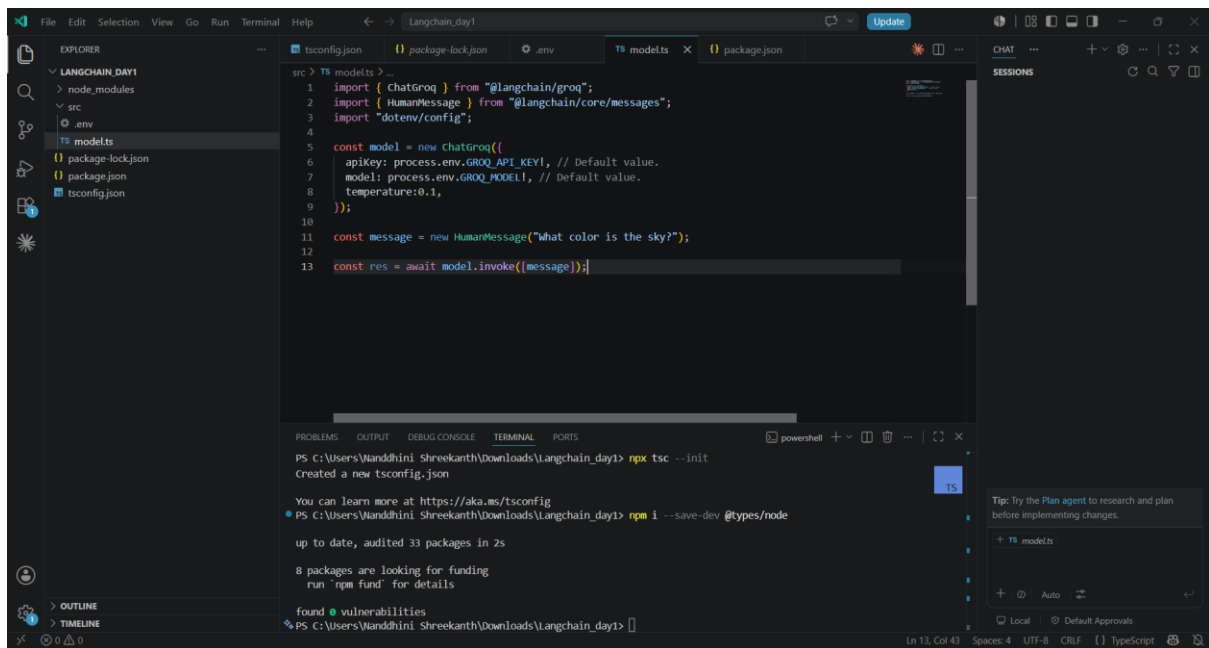
```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm tsc --init
Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm i --save-dev @types/node
up to date, audited 33 packages in 2s

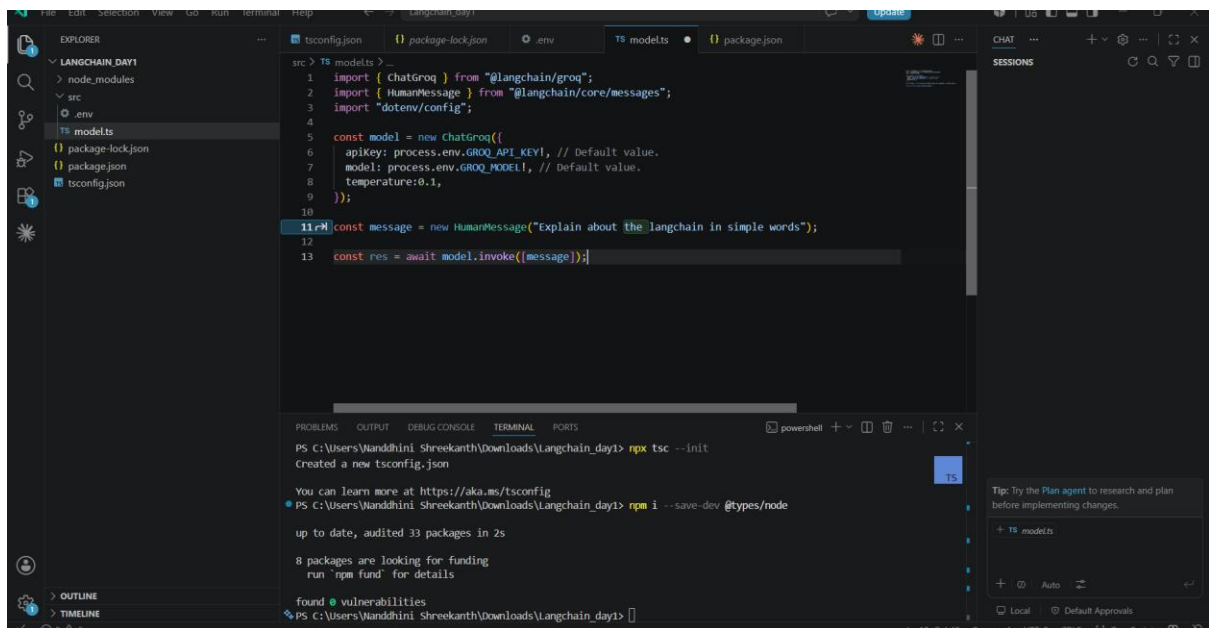
8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

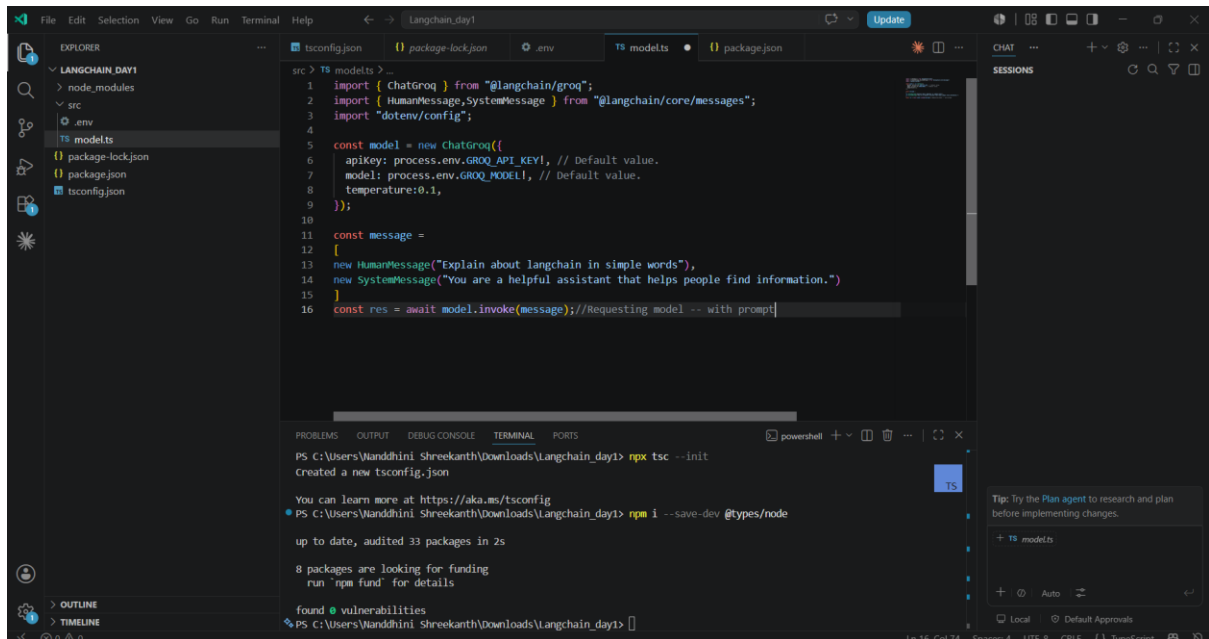
Checking the error is gone in model.ts



Change the human message in model.ts



Add SystemMessage in Line 2 in model.js and give new SystemMessage in line 14 and remove [] in line 16



The screenshot shows the VS Code editor with the `model.ts` file open. The code is as follows:

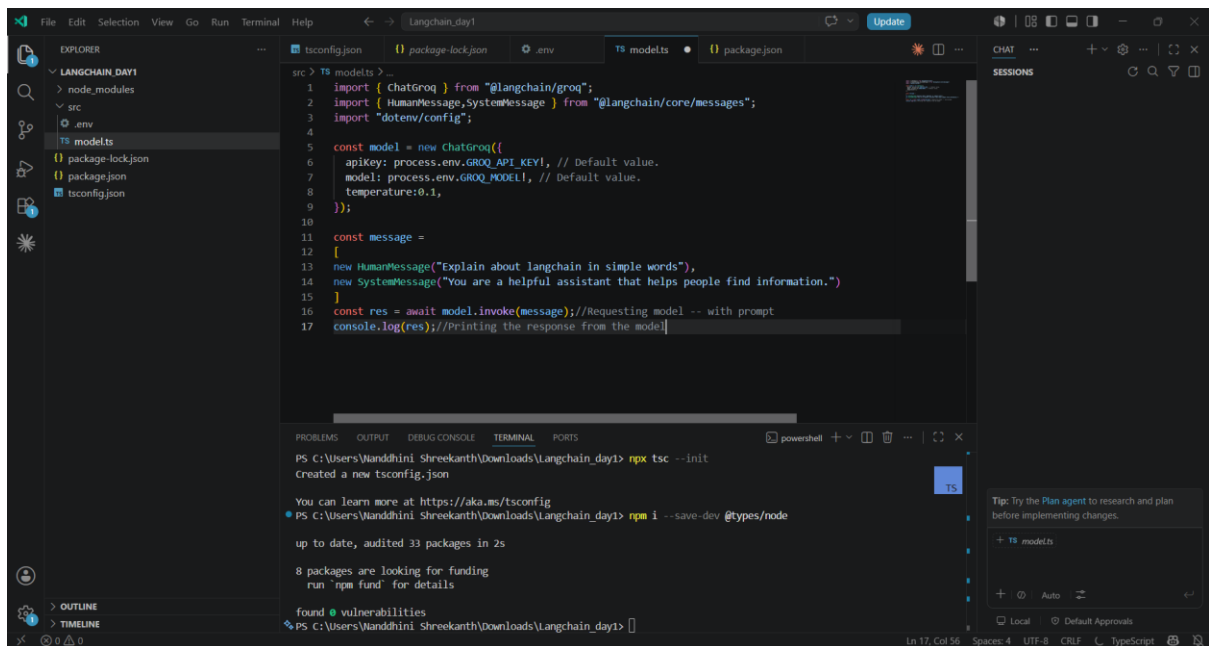
```
src > TS models > ...
1 import { ChatGPT } from "@langchain/groq";
2 import { HumanMessage, SystemMessage } from "@langchain/core/messages";
3 import "dotenv/config";
4
5 const model = new ChatGPT({
6   apiKey: process.env.GROQ_API_KEY!, // Default value.
7   model: process.env.GROQ_MODEL!, // Default value.
8   temperature: 0.1,
9 });
10
11 const message =
12 [
13   new HumanMessage("Explain about langchain in simple words"),
14   new SystemMessage("You are a helpful assistant that helps people find information.")
15 ]
16 const res = await model.invoke(message); // Requesting model -- with prompt
```

The terminal output shows the following commands and results:

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npx tsc --init
Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm i --save-dev @types/node
up to date, audited 33 packages in 2s
8 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

For printing the response console.log is added in line 17 in model.ts

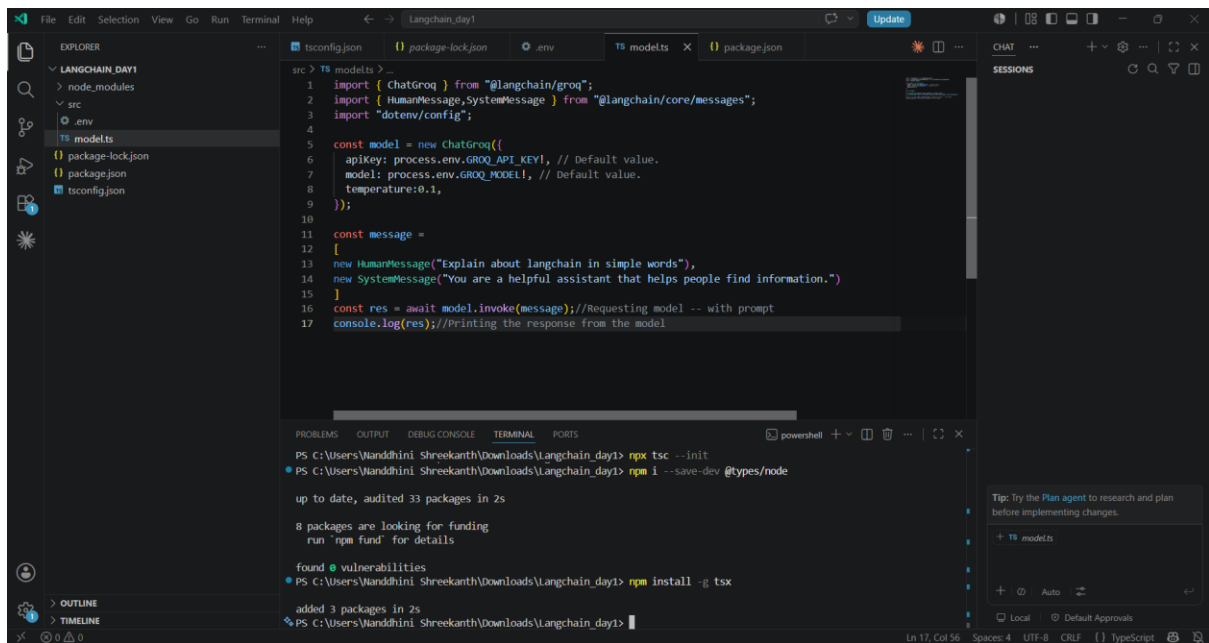


The screenshot shows the VS Code editor with the `model.ts` file open. The code is as follows:

```
src > TS models > ...
1 import { ChatGPT } from "@langchain/groq";
2 import { HumanMessage, SystemMessage } from "@langchain/core/messages";
3 import "dotenv/config";
4
5 const model = new ChatGPT({
6   apiKey: process.env.GROQ_API_KEY!, // Default value.
7   model: process.env.GROQ_MODEL!, // Default value.
8   temperature: 0.1,
9 });
10
11 const message =
12 [
13   new HumanMessage("Explain about langchain in simple words"),
14   new SystemMessage("You are a helpful assistant that helps people find information.")
15 ]
16 const res = await model.invoke(message); // Requesting model -- with prompt
17 console.log(res); // Printing the response from the model
```

The terminal output is the same as in the previous screenshot, showing the successful installation of dependencies and the creation of the `tsconfig.json` file.

Executing the typescript file npm install -g tsx in terminal



The screenshot shows the VS Code interface with a project named 'Langchain_day1'. The Explorer panel on the left shows the file structure: 'LANGCHAIN_DAY1' containing 'node_modules', 'src', '.env', 'package-lock.json', 'tsconfig.json', and 'TS models'. The 'TS models' file is open in the editor, showing a TypeScript script that imports 'ChatGroq' from '@langchain/groq' and uses it to create a chat model and send a message. The terminal window at the bottom shows the following commands and output:

```
PS C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1> npm tsc --init
PS C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1> npm i --save-dev @types/node

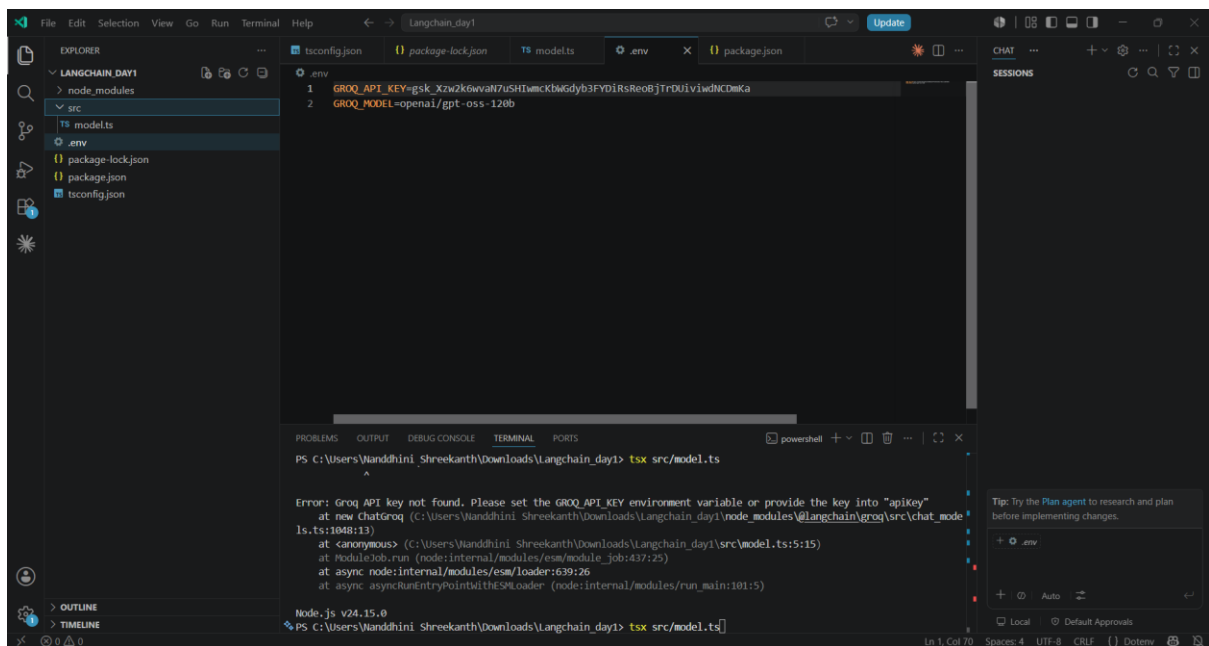
up to date, audited 33 packages in 2s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1> npm install -g tsx

added 3 packages in 2s
PS C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1>
```

Create .env file outside the src folder and delete the .env file which is created inside the src folder



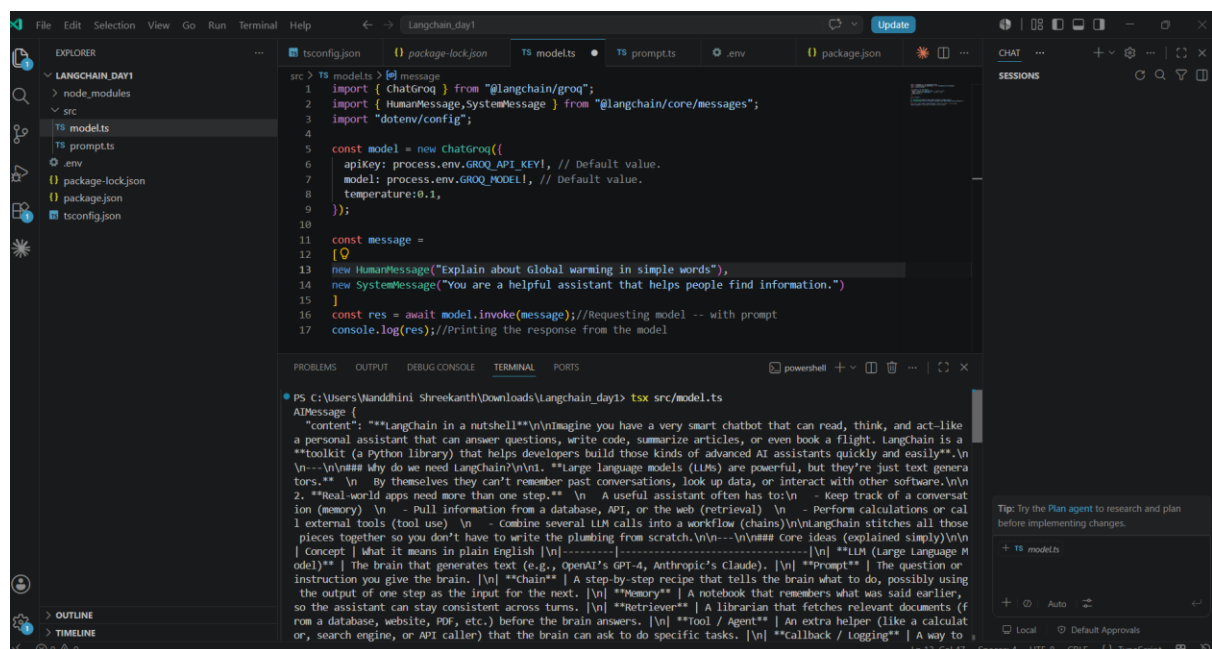
The screenshot shows the VS Code interface with the same project. The Explorer panel shows the file structure, and the '.env' file is now visible in the root of the project. The terminal window shows the following commands and output:

```
PS C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1> tsx src/model.ts

Error: Groq API key not found. Please set the GROQ_API_KEY environment variable or provide the key into "apiKey"
at new ChatGroq (C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1\node_modules\@langchain\groq\src\chat_model.ts:1048:13)
at <anonymous> (C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1\src\model.ts:5:15)
at ModuleJob.run (node:internal/modules/esm/module_job:437:25)
at async node:internal/modules/esm/loader:639:26
at async RunEntryPoint.run (node:internal/modules/run_main:101:5)

Node.js v24.15.0
PS C:\Users\Vanddhini Shreekanth\Downloads\Langchain_day1> tsx src/model.ts]
```

Execute tsx src/model.ts



```
src > ts models > message
1 import { ChatGPT } from "@langchain/groq";
2 import { HumanMessage, SystemMessage } from "@langchain/core/messages";
3 import "dotenv/config";
4
5 const model = new ChatGPT({
6   apiKey: process.env.GROQ_API_KEY, // Default value.
7   model: process.env.GROQ_MODEL, // Default value.
8   temperature: 0.1,
9 });
10
11 const message =
12   [
13     new HumanMessage("Explain about Global warming in simple words"),
14     new SystemMessage("You are a helpful assistant that helps people find information.")
15   ]
16 const res = await model.invoke(message); // Requesting model -- with prompt
17 console.log(res); // Printing the response from the model
```

```
PS C:\Users\Hindini\Shreekanth\Downloads\langchain_day1> tsx src/model.ts
AIMessage {
  "content": "***LangChain in a nutshell**\n\nImagine you have a very smart chatbot that can read, think, and act—like a personal assistant that can answer questions, write code, summarize articles, or even book a flight. LangChain is a **toolkit (a Python library) that helps developers build those kinds of advanced AI assistants quickly and easily**.\n\n---\n\n### Why do we need LangChain?\n\n1. **Large language models (LLMs) are powerful, but they're just text generators.** \n By themselves they can't remember past conversations, look up data, or interact with other software.\n\n2. **Real-world apps need more than one step.** \n A useful assistant often has to:\n - Keep track of a conversation (memory) \n - Pull information from a database, API, or the web (retrieval) \n - Perform calculations or call external tools (tool use) \n - Combine several LLM calls into a workflow (chains)\n\nLangChain stitches all those pieces together so you don't have to write the plumbing from scratch.\n\n---\n\n### Core ideas (explained simply)\n\n| Concept | What it means in plain English |\n|-----|-----|\n\n| **LLM (Large Language Model)** | The brain that generates text (e.g., OpenAI's GPT-4, Anthropic's Claude). |\n| **Prompt** | The question or instruction you give the brain. |\n| **Chain** | A step-by-step recipe that tells the brain what to do, possibly using the output of one step as the input for the next. |\n| **Memory** | A notebook that remembers what was said earlier, so the assistant can stay consistent across turns. |\n| **Retriever** | A librarian that fetches relevant documents (from a database, website, PDF, etc.) before the brain answers. |\n| **Tool / Agent** | An extra helper (like a calculator, search engine, or API caller) that the brain can ask to do specific tasks. |\n| **Callback / Logging** | A way to watch what's happening inside the recipe (useful for debugging or showing progress). |\n\n---\n\n### How you would use LangChain (a simple example)\n\n1. **Pick an LLM** – e.g., `OpenAIChat(model="gpt-4")`\n2. **Add a Retriever** – e.g., a vector-search over your company's knowledge base.\n3. **Create a Chain** – “First fetch the most relevant docs, then ask the LLM to answer the user's question using those docs.”\n4. **Run the Chain** – Feed the user's question, get a nicely formatted answer.\n\n```\npython\nfrom langchain.llms import OpenAI\nfrom langchain.vectorstores import FAISS\nfrom langchain.chains import RetrievalQA\n\n#
```

The below is the response in terminal AIMessage {

"content": "***LangChain in a nutshell**\n\nImagine you have a very smart chatbot that can read, think, and act—like a personal assistant that can answer questions, write code, summarize articles, or even book a flight. LangChain is a **toolkit (a Python library) that helps developers build those kinds of advanced AI assistants quickly and easily**.\n\n---\n\n### Why do we need LangChain?\n\n1. **Large language models (LLMs) are powerful, but they're just text generators.** \n By themselves they can't remember past conversations, look up data, or interact with other software.\n\n2. **Real-world apps need more than one step.** \n A useful assistant often has to:\n - Keep track of a conversation (memory) \n - Pull information from a database, API, or the web (retrieval) \n - Perform calculations or call external tools (tool use) \n - Combine several LLM calls into a workflow (chains)\n\nLangChain stitches all those pieces together so you don't have to write the plumbing from scratch.\n\n---\n\n### Core ideas (explained simply)\n\n| Concept | What it means in plain English |\n|-----|-----|\n\n| **LLM (Large Language Model)** | The brain that generates text (e.g., OpenAI's GPT-4, Anthropic's Claude). |\n| **Prompt** | The question or instruction you give the brain. |\n| **Chain** | A step-by-step recipe that tells the brain what to do, possibly using the output of one step as the input for the next. |\n| **Memory** | A notebook that remembers what was said earlier, so the assistant can stay consistent across turns. |\n| **Retriever** | A librarian that fetches relevant documents (from a database, website, PDF, etc.) before the brain answers. |\n| **Tool / Agent** | An extra helper (like a calculator, search engine, or API caller) that the brain can ask to do specific tasks. |\n| **Callback / Logging** | A way to watch what's happening inside the recipe (useful for debugging or showing progress). |\n\n---\n\n### How you would use LangChain (a simple example)\n\n1. **Pick an LLM** – e.g., `OpenAIChat(model="gpt-4")`\n2. **Add a Retriever** – e.g., a vector-search over your company's knowledge base.\n3. **Create a Chain** – “First fetch the most relevant docs, then ask the LLM to answer the user's question using those docs.”\n4. **Run the Chain** – Feed the user's question, get a nicely formatted answer.\n\n```\npython\nfrom langchain.llms import OpenAI\nfrom langchain.vectorstores import FAISS\nfrom langchain.chains import RetrievalQA\n\n#

```

LLM\nllm = OpenAI(model="gpt-4")\n\n# 2 Retriever (search over
embeddings)\nvector_store = FAISS.load_local("my_docs_index")\nretriever =
vector_store.as_retriever(search_kwargs={"k": 3})\n\n# 3 Chain that ties them
together\nqa_chain = RetrievalQA.from_chain_type(\n    llm=llm,\n    chain_type="stuff", #
simple "stuff" chain\n    retriever=retriever,\n)\n\n# 4 Ask a question\nanswer =
qa_chain.run("How do we handle refunds?")\nprint(answer)\n` ``\n\n*All of that is just a few
lines of code, thanks to LangChain's pre-built pieces.*\n\n---\n\n### What you can build with
LangChain\n\n| Use-case | What LangChain helps you do |\n|-----|-----|\n|
**Customer-support bot** | Remember the user's ticket ID, look up past chats, fetch relevant
policy docs, and generate a helpful reply. |\n| **Research assistant** | Search a large corpus of
scientific papers, summarize findings, and cite sources automatically. |\n| **Data-analysis
helper** | Retrieve a CSV file, ask the LLM to run a calculation, and get back a chart or
explanation. |\n| **Automation agent** | Let the LLM decide to call an external API (e.g., booking
a meeting) and execute it safely. |\n| **Personal productivity** | Build a "second brain" that
stores notes, reminds you of tasks, and drafts emails. |\n\n---\n\n### Why many developers
love it\n\n- **Modular** – You can swap in any LLM, any vector store, any tool, without rewriting
the whole app.\n- **Ready-made patterns** – Common recipes (retrieval-augmented
generation, tool-using agents, multi-step reasoning) are already implemented.\n- **Community
& docs** – Lots of examples, tutorials, and a growing ecosystem of extensions.\n-
**Open-source** – Free to use, inspect, and extend.\n\n---\n\n### TL;DR (the ultra-simple
version)\n\n> **LangChain = a set of building blocks that let you turn a raw language model (like
GPT-4) into a full-featured AI assistant that can remember, look up information, and interact
with other software, all with just a few lines of code.**\n\nIf you ever wanted to make a chatbot
that does more than "talk," LangChain is the shortcut that handles the plumbing so you can
focus on the actual conversation logic.",

```

```

"additional_kwargs": {},

"response_metadata": {

    "tokenUsage": {

        "completionTokens": 1174,

        "promptTokens": 92,

        "totalTokens": 1266

    },

    "finish_reason": "stop",

    "id": "chatcmpl-47dd9c6d-1422-425e-a851-75521323f46c",

    "object": "chat.completion",

    "created": 1782908115,

    "model": "openai/gpt-oss-120b",

    "usage": {

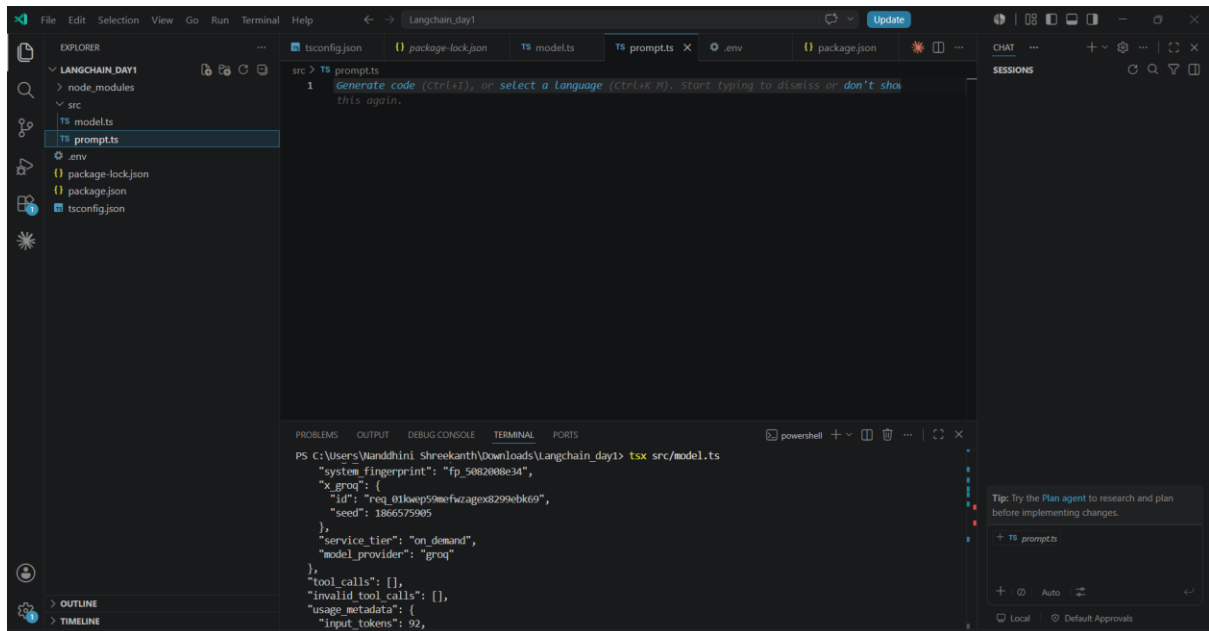
        "queue_time": 1.5104083,

```

```
"prompt_tokens": 92,
"prompt_time": 0.027837413,
"completion_tokens": 1174,
"completion_time": 2.466643021,
"total_tokens": 1266,
"total_time": 2.494480434,
"completion_tokens_details": {
  "reasoning_tokens": 58
}
},
"usage_breakdown": null,
"system_fingerprint": "fp_93703442d9",
"x_groq": {
  "id": "req_01kwesnjh3e9f91fj7g285gykq",
  "seed": 750678257
},
"service_tier": "on_demand",
"model_provider": "groq"
},
"tool_calls": [],
"invalid_tool_calls": [],
"usage_metadata": {
  "input_tokens": 92,
  "output_tokens": 1174,
  "total_tokens": 1266
}
}
```

2. Prompt Template

Create a file as prompt.ts inside the src folder



Copy the below code in prompt.ts

```
const message = SystemMessagePromptTemplate.fromTemplate("{text}");

const chatPrompt = ChatPromptTemplate.fromMessages([

  ["ai", "You are a helpful assistant."],

  message,

]);

const formattedChatPrompt = await chatPrompt.invoke({

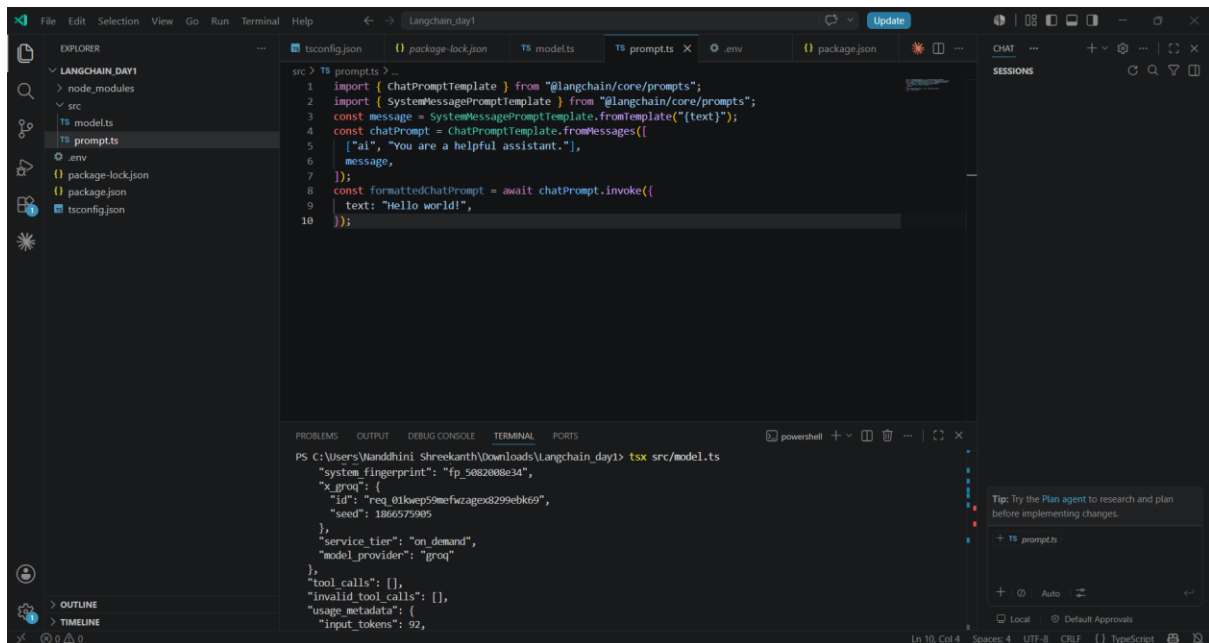
  text: "Hello world!",

});
```

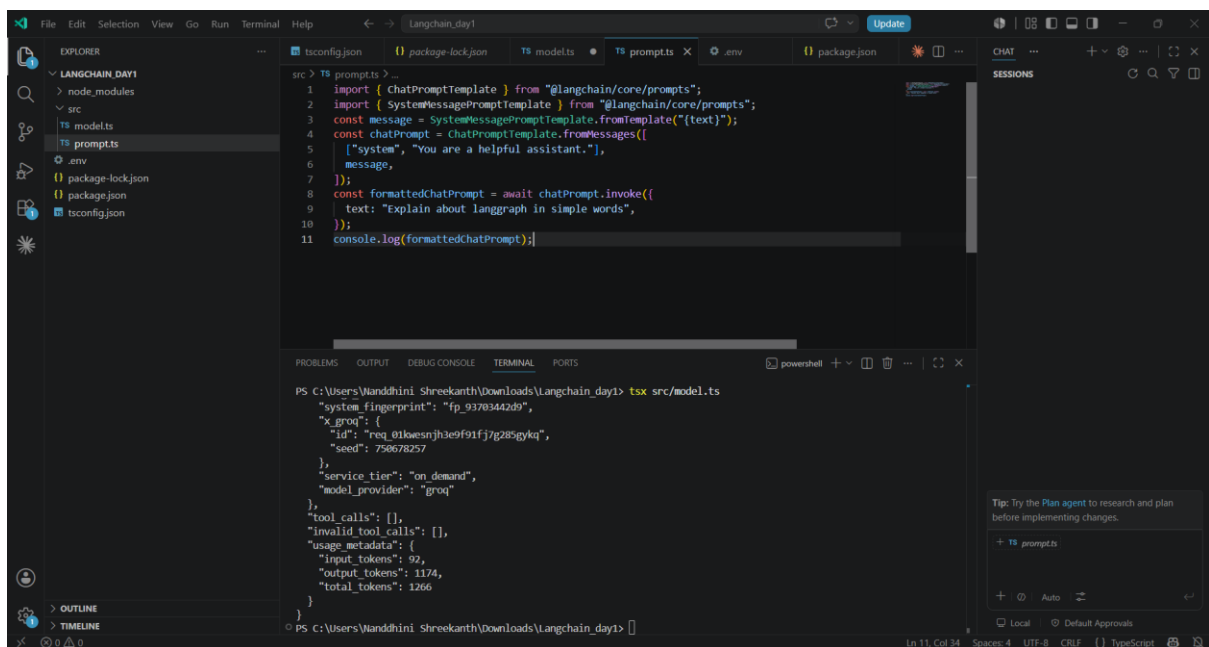
And add the import file on the first two lines

```
import { ChatPromptTemplate } from "@langchain/core/prompts";
```

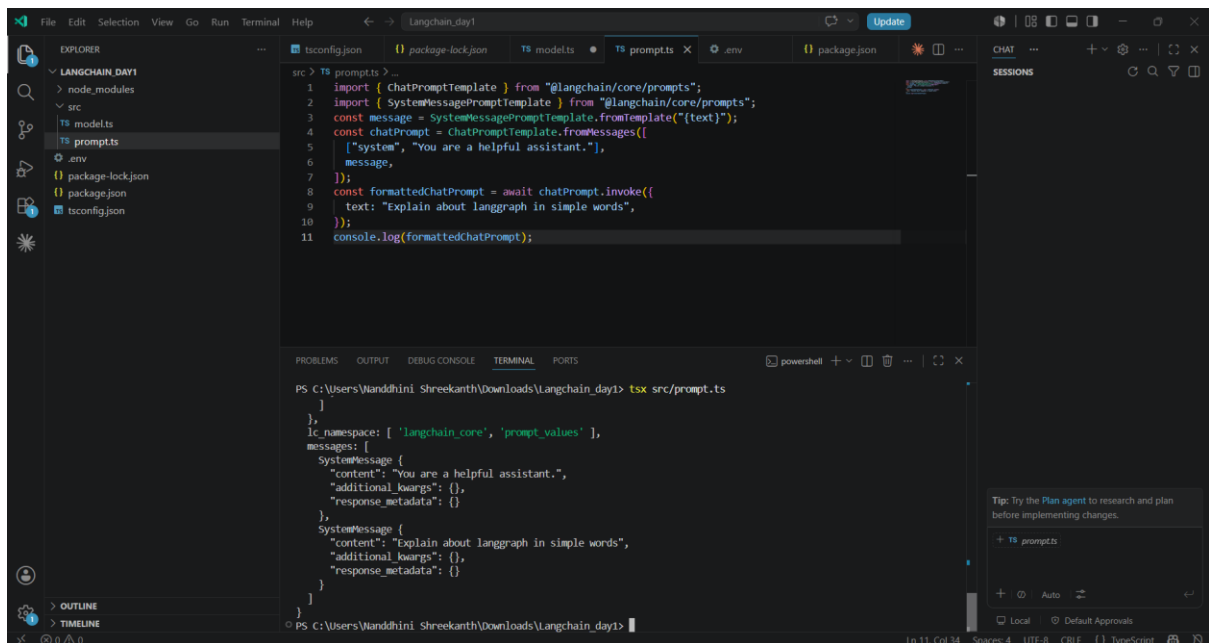
```
import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
```



Adding the text message as “Explain about langgraph in simple words” and added console.log for printing the message



Execute tsx src/prompt.ts and verify the response in terminal



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'LANGCHAIN_DAY1' with a 'src' directory containing 'prompt.ts'. The code editor shows the contents of 'prompt.ts', which imports 'ChatPromptTemplate' and 'SystemMessagePromptTemplate' from '@langchain/core/prompts', creates a chat prompt with two messages, and logs the formatted chat prompt. The terminal shows the command 'tsx src/prompt.ts' being executed, and the output is a JSON object representing the formatted chat prompt.

```
1 import { ChatPromptTemplate } from "@langchain/core/prompts";
2 import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
3 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
4 const chatPrompt = ChatPromptTemplate.fromMessages([
5   ["system", "You are a helpful assistant."],
6   message,
7 ]);
8 const formattedChatPrompt = await chatPrompt.invoke({
9   text: "Explain about langgraph in simple words",
10 });
11 console.log(formattedChatPrompt);
```

```
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1> tsx src/prompt.ts
{
  lc_namespace: [ 'langchain_core', 'prompt_values' ],
  messages: [
    SystemMessage {
      content: "You are a helpful assistant.",
      additional_kwargs: {},
      response_metadata: {}
    },
    SystemMessage {
      content: "Explain about langgraph in simple words",
      additional_kwargs: {},
      response_metadata: {}
    }
  ]
}
```

```
ChatPromptValue {
  lc_serializable: true,
  lc_kwargs: {
    messages: [
      SystemMessage {
        "content": "You are a helpful assistant.",
        "additional_kwargs": {},
        "response_metadata": {}
      },
      SystemMessage {
        "content": "Explain about langgraph in simple words",
        "additional_kwargs": {},
        "response_metadata": {}
      }
    ]
  },
  lc_namespace: [ 'langchain_core', 'prompt_values' ],
  messages: [
    SystemMessage {
```



```

"content": "You are a helpful assistant.",

"additional_kwargs": {},

"response_metadata": {}
},

SystemMessage {

"content": "Explain about langgraph in simple words",

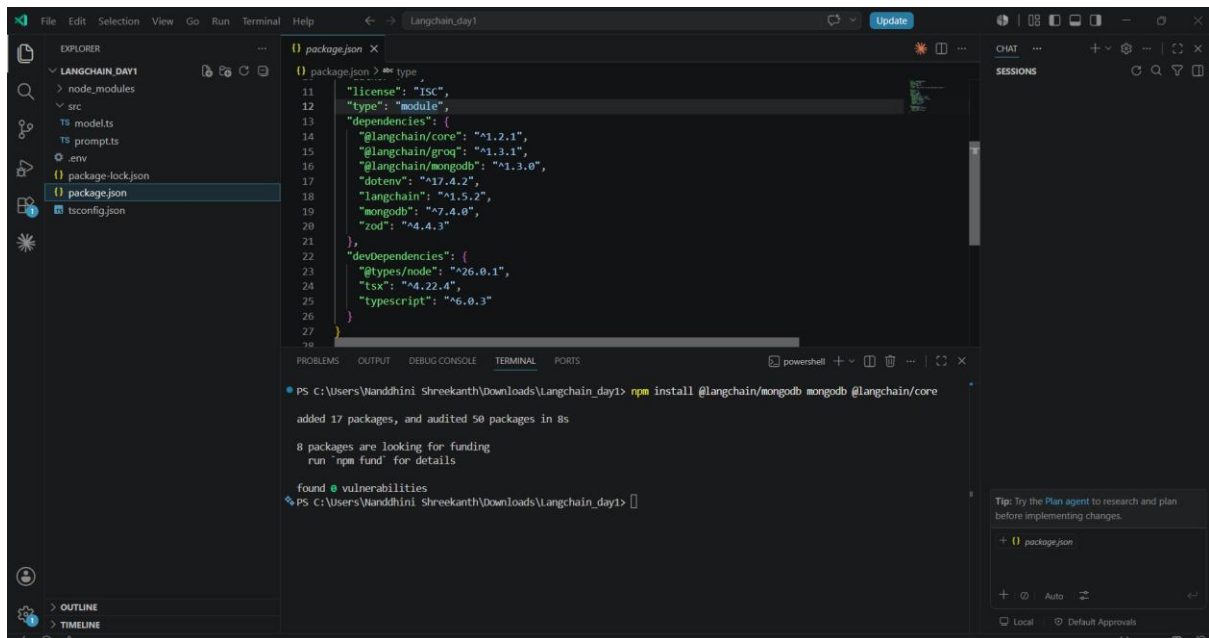
"additional_kwargs": {},

"response_metadata": {}
}
]
}

```

3. Tools and Integrations: -

Installation of Mongo Db and execute in terminal for `npm install @langchain/mongodb mongodb @langchain/core` and it is available in package.json



Installation of Mistral and execute in terminal for npm install @langchain/mistralai
@langchain/core

The screenshot shows a VS Code editor with a file explorer on the left, a package.json file open in the center, and a terminal at the bottom. The package.json file has the following content:

```
11 "license": "ISC",
12 "type": "module",
13 "dependencies": {
14   "@langchain/core": "^1.2.1",
15   "@langchain/groq": "^1.3.1",
16   "@langchain/mistralai": "^1.2.0",
17   "@langchain/mongodb": "^1.3.0",
18   "dotenv": "^17.4.2",
19   "langchain": "^1.5.2",
20   "mongodb": "^7.4.0",
21   "zod": "^4.4.3"
22 },
23 "devDependencies": {
24   "@types/node": "^26.0.1",
25   "tsx": "^4.22.4",
26   "typescript": "^6.0.3"
27 }
```

The terminal shows the following commands and output:

```
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1> npm install @langchain/mongodb mongodb @langchain/core
added 17 packages, and audited 50 packages in 8s
8 packages are looking for funding
run npm fund for details
found 0 vulnerabilities
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1> npm install @langchain/mistralai @langchain/core
added 4 packages, and audited 54 packages in 14s
8 packages are looking for funding
run npm fund for details
found 0 vulnerabilities
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1>
```

Execute in terminal for npm install mongodb@^6.20.0

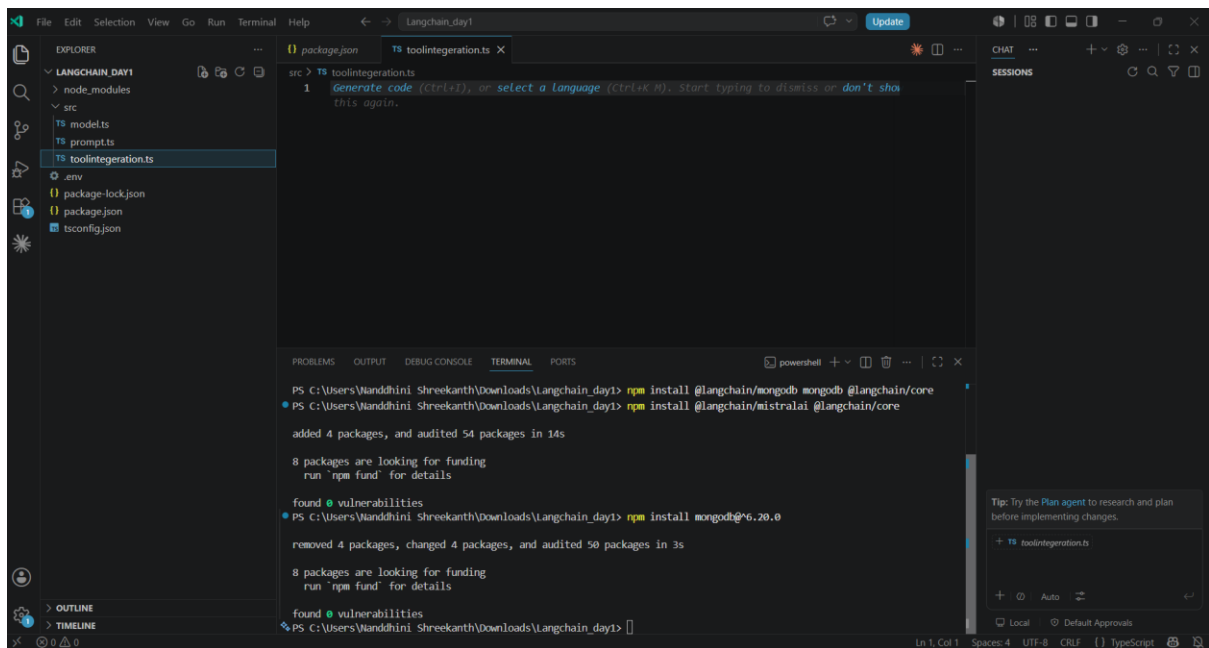
The screenshot shows a VS Code editor with a file explorer on the left, a package.json file open in the center, and a terminal at the bottom. The package.json file has the following content:

```
11 "license": "ISC",
12 "type": "module",
13 "dependencies": {
14   "@langchain/core": "^1.2.1",
15   "@langchain/groq": "^1.3.1",
16   "@langchain/mistralai": "^1.2.0",
17   "@langchain/mongodb": "^1.3.0",
18   "dotenv": "^17.4.2",
19   "langchain": "^1.5.2",
20   "mongodb": "^6.21.0",
21   "zod": "^4.4.3"
22 },
23 "devDependencies": {
24   "@types/node": "^26.0.1",
25   "tsx": "^4.22.4",
26   "typescript": "^6.0.3"
27 }
```

The terminal shows the following commands and output:

```
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1> npm install @langchain/mongodb mongodb @langchain/core
added 4 packages, and audited 54 packages in 14s
8 packages are looking for funding
run npm fund for details
found 0 vulnerabilities
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1> npm install mongodb@^6.20.0
removed 4 packages, changed 4 packages, and audited 50 packages in 3s
8 packages are looking for funding
run npm fund for details
found 0 vulnerabilities
PS C:\Users\Iandhini Shreekanth\Downloads\Langchain_day1>
```

Creating a file under src as toolintegration.ts



Copy the import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";

import { OpenAIEmbeddings } from "@langchain/openai";

import { MongoClient } from "mongodb";

const client = new MongoClient(process.env.MONGODB_ATLAS_URI!);

const collection = client

.db(process.env.MONGODB_ATLAS_DB_NAME)

.collection(process.env.MONGODB_ATLAS_COLLECTION_NAME);

const embeddings = new OpenAIEmbeddings({

model: "text-embedding-3-small",

});

const vectorStore = new MongoDBAtlasVectorSearch(embeddings, {

collection,

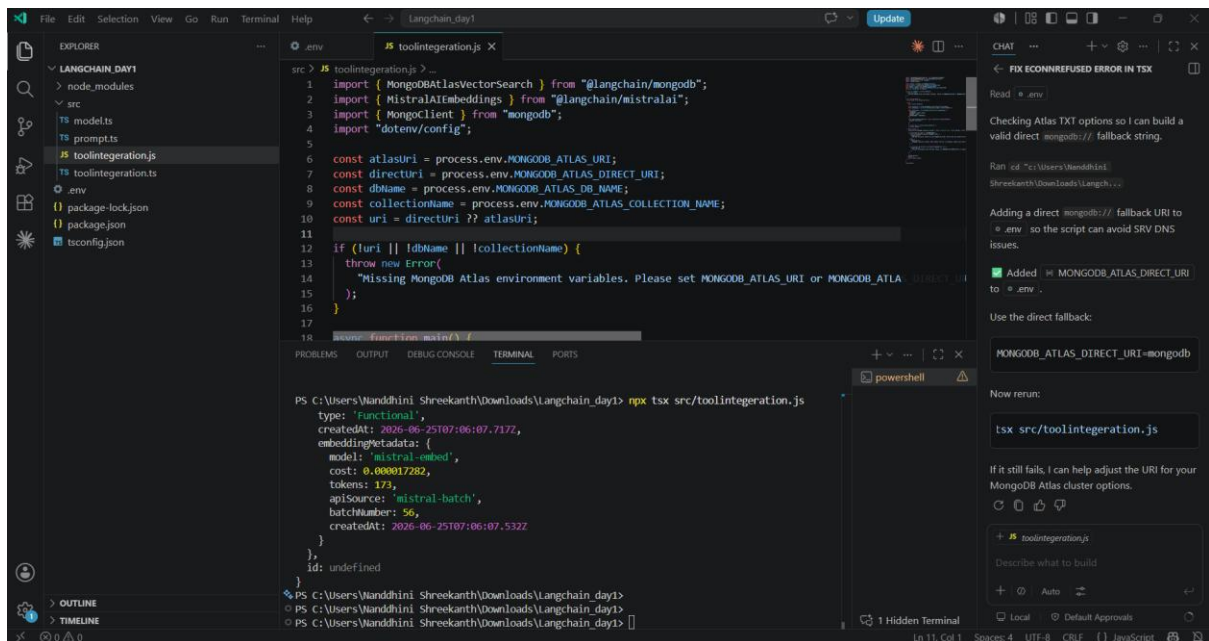
indexName: "vector_index", // Defaults to "default"

textKey: "text", // Defaults to "text"

embeddingKey: "embedding", // Defaults to "embedding"

});

This in toolintegration.js



The screenshot shows a VS Code editor with the file `toolintegration.js` open. The file contains the following code:

```
1 import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";
2 import { MistralAIEmbeddings } from "@langchain/mistralai";
3 import { MongoClient } from "mongodb";
4 import "dotenv/config";
5
6 const atlasUri = process.env.MONGODB_ATLAS_URI;
7 const directUri = process.env.MONGODB_ATLAS_DIRECT_URI;
8 const dbName = process.env.MONGODB_ATLAS_DB_NAME;
9 const collectionName = process.env.MONGODB_ATLAS_COLLECTION_NAME;
10 const uri = directUri ?? atlasUri;
11
12 if (!uri || !dbName || !collectionName) {
13   throw new Error(
14     "Missing MongoDB Atlas environment variables. Please set MONGODB_ATLAS_URI or MONGODB_ATLAS_DIRECT_URI, MONGODB_ATLAS_DB_NAME, and MONGODB_ATLAS_COLLECTION_NAME."
15   );
16 }
17
18 async function main() {
19   // ...
20 }
```

The terminal window shows the output of running `npm run tsx src/toolintegration.js`:

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm run tsx src/toolintegration.js
type: 'Functional',
createdat: 2026-06-25T07:06:07.717Z,
embeddingMetadata: {
  model: 'mistral-embed',
  cost: 0.000017282,
  tokens: 173,
  apiSource: 'mistral-batch',
  batchSize: 56,
  createdAt: 2026-06-25T07:06:07.532Z
},
id: undefined
```

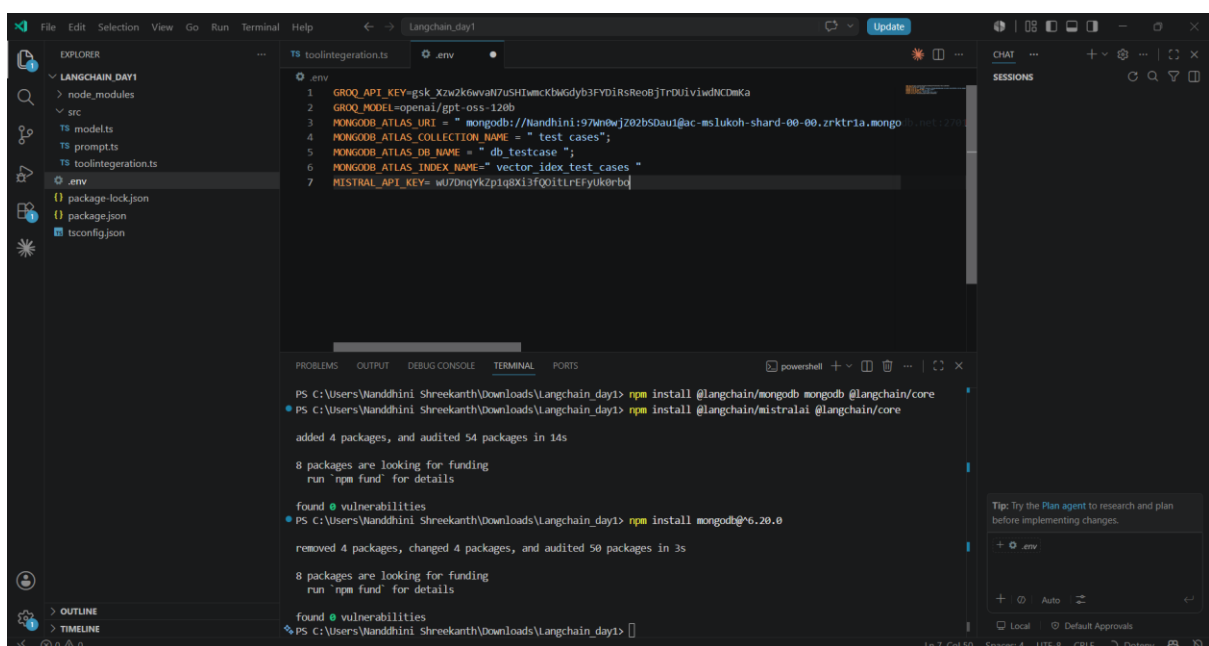
Add `MONGODB_ATLAS_URI = "mongodb://Nandhini:97Wn0wjZ02bSDau1@ac-mslukoh-shard-00-00.zrkr1a.mongodb.net:27017,ac-mslukoh-shard-00-01.zrkr1a.mongodb.net:27017,ac-mslukoh-shard-00-02.zrkr1a.mongodb.net:27017/?ssl=true&replicaSet=atlas-brxbho-shard-0&authSource=admin&appName=Cluster0"`;

`MONGODB_ATLAS_COLLECTION_NAME = " test cases";`

`MONGODB_ATLAS_DB_NAME = " db_testcase ";`

`MONGODB_ATLAS_INDEX_NAME=" vector_idex_test_cases "`

`MISTRAL_API_KEY= wU7DnqYkZp1q8Xi3fQOitLrEFyUk0rbo`



The screenshot shows a VS Code editor with the `.env` file open. The file contains the following environment variables:

```
1 GROQ_API_KEY=gsk_Xzw2K6wvaN7uSHImcKbGdyb3FYDiRSReoBjTrDUiViwdNCDmKa
2 GROQ_MODEL=openai/gpt-oss-120b
3 MONGODB_ATLAS_URI = "mongodb://Nandhini:97Wn0wjZ02bSDau1@ac-mslukoh-shard-00-00.zrkr1a.mongodb.net:27017,ac-mslukoh-shard-00-01.zrkr1a.mongodb.net:27017,ac-mslukoh-shard-00-02.zrkr1a.mongodb.net:27017/?ssl=true&replicaSet=atlas-brxbho-shard-0&authSource=admin&appName=Cluster0"
4 MONGODB_ATLAS_COLLECTION_NAME = " test cases";
5 MONGODB_ATLAS_DB_NAME = " db_testcase ";
6 MONGODB_ATLAS_INDEX_NAME=" vector_idex_test_cases "
7 MISTRAL_API_KEY= wU7DnqYkZp1q8Xi3fQOitLrEFyUk0rbo
```

The terminal window shows the output of running `npm install @langchain/mongodb mongodb @langchain/core` and `npm install @langchain/mistralai @langchain/core`:

```
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install @langchain/mongodb mongodb @langchain/core
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install @langchain/mistralai @langchain/core

added 4 packages, and audited 54 packages in 14s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1> npm install mongodb@6.20.0

removed 4 packages, changed 4 packages, and audited 50 packages in 3s

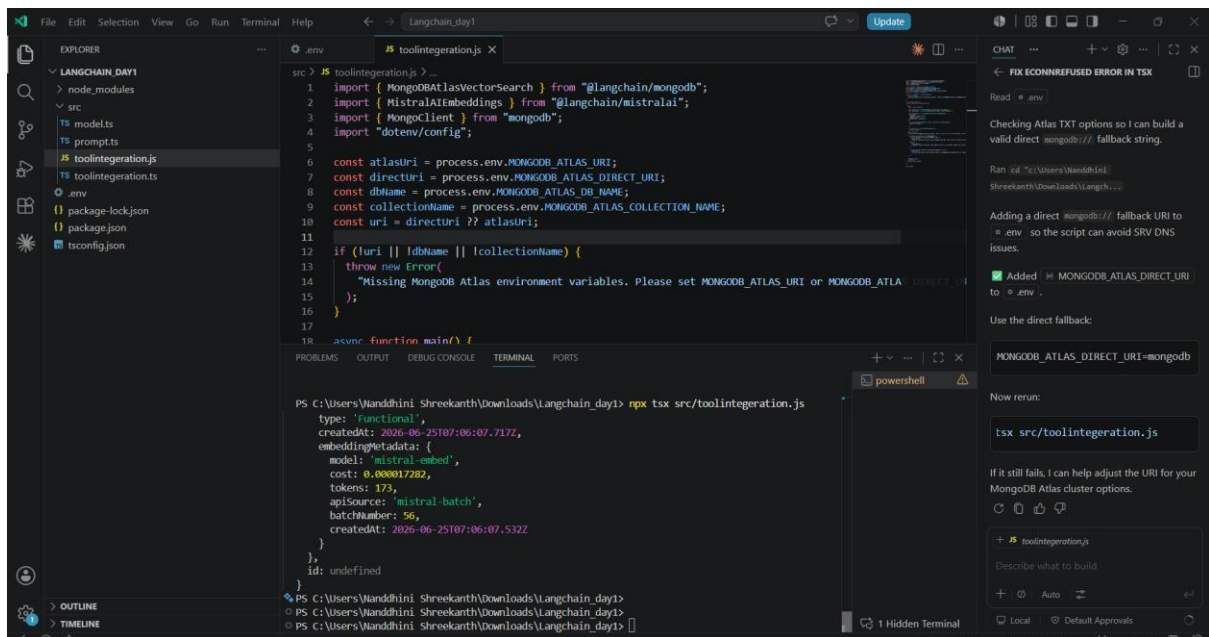
8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\Vandhini Shreekanth\Downloads\Langchain_day1>
```

Copy the entire line `import { MistralAIEmbeddings } from "@langchain/mistralai";` from [Embedding model integrations - Docs by LangChain](#) and replace in 2nd line in `toolintegration.js`

Replace OpenAI embedding with `MistralAIEmbeddings` and change the model as `model: "mistral-embed"`,

Add 4th line as `import "dotenv/config";` and add `!` in 8th and 9th line and add the vector index name as well



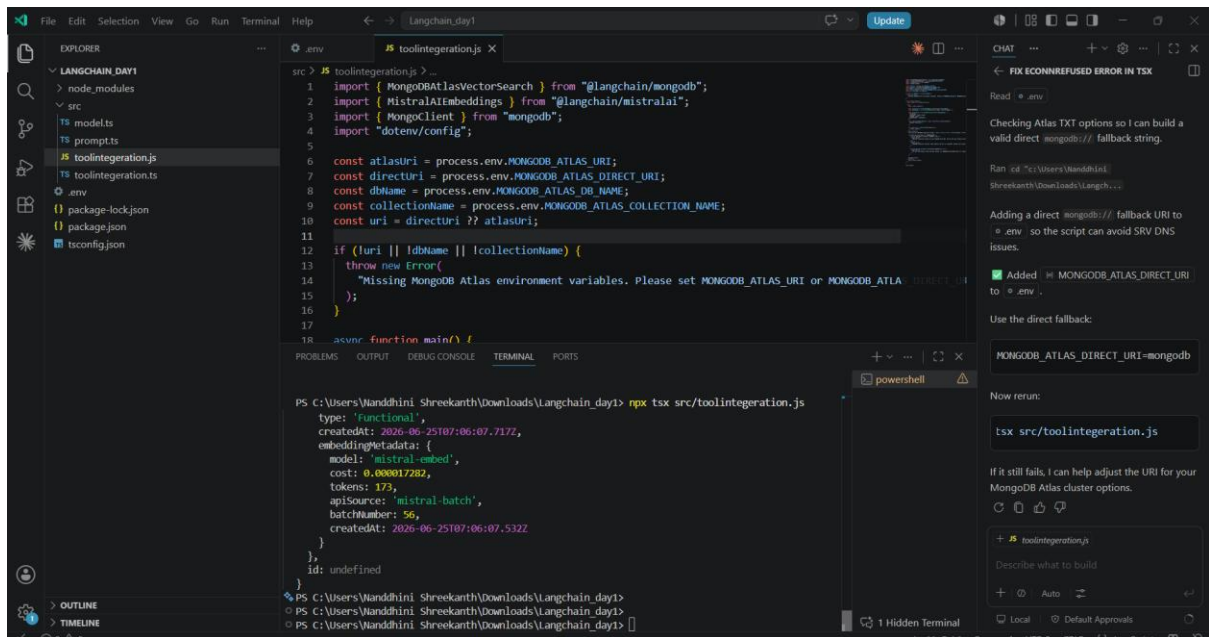
Copy this line and add it in `toolintegration.js`

Query directly

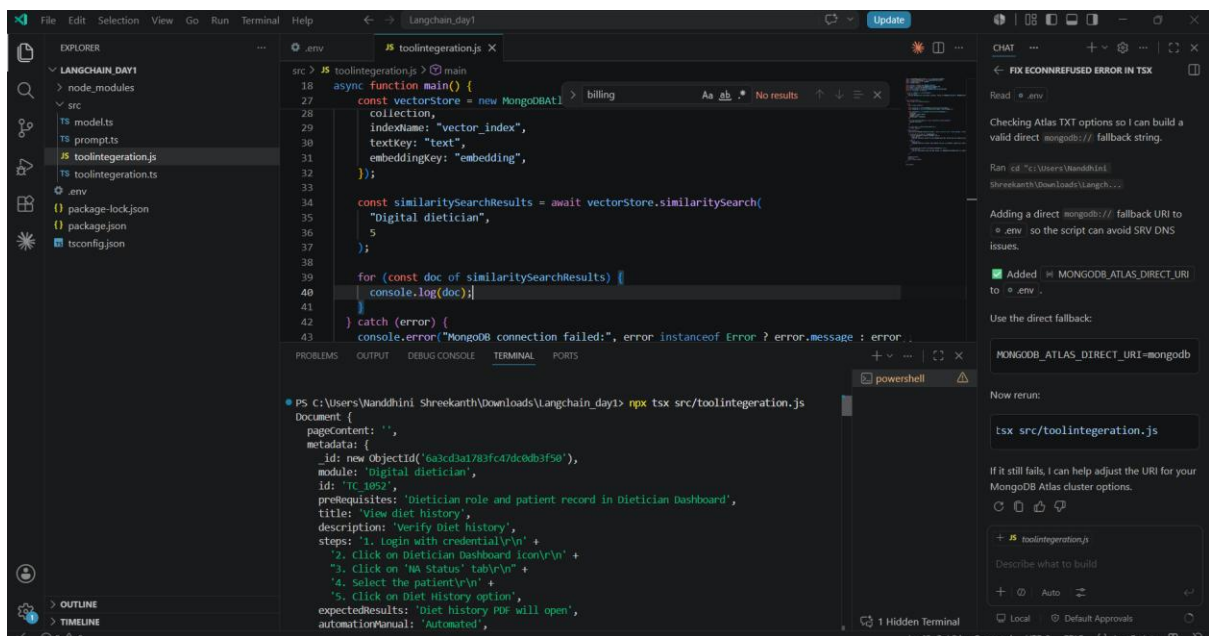
Performing a simple similarity search can be done as follows:

```
const similaritySearchResults = await vectorStore.similaritySearch("bi

for (const doc of similaritySearchResults) {
  console.log(`* ${doc.pageContent} [${JSON.stringify(doc.metadata, null)}]`)
}
```



Execute `tsx src/toolintegration.js`



Response:

Document {

pageContent: '',

metadata: {

_id: new ObjectId('6a3cd3a1783fc47dc0db3f50'),

module: 'Digital dietician',

id: 'TC_1052',

preRequisites: 'Dietician role and patient record in Dietician Dashboard',

```
title: 'View diet history',
description: 'Verify Diet history',
steps: '1. Login with credential\r\n' +
  '2. Click on Dietician Dashboard icon\r\n' +
  '3. Click on 'NA Status' tab\r\n' +
  '4. Select the patient\r\n' +
  '5. Click on Diet History option',
expectedResults: 'Diet history PDF will open',
automationManual: 'Automated',
priority: 'P3',
createdBy: 'Prabakaran',
createdDate: 45044,
lastModifiedDate: 45119,
linkedStories: 'HC-012, HC-054',
risk: 'Low',
version: 'v15.0',
type: 'UI',
createdAt: 2026-06-25T07:07:13.060Z,
embeddingMetadata: {
  model: 'mistral-embed',
  cost: 0.000015316,
  tokens: 153,
  apiSource: 'mistral-batch',
  batchNumber: 103,
  createdAt: 2026-06-25T07:07:12.127Z
},
id: undefined
}
Document {
  pageContent: '',
```

```
metadata: {
  _id: new ObjectId('6a3cd3a1783fc47dc0db3f4f'),
  module: 'Digital dietician',
  id: 'TC_1051',
  preRequisites: 'Dietician role and patient record in Dietician Dashboard',
  title: 'View Diet sticker',
  description: 'Verify Diet sticker',
  steps: '1. Login with credential\r\n' +
    '2. Click on Dietician Dashboard icon\r\n' +
    '3. Click on 'NA Status' tab\r\n' +
    '4. Select the patient\r\n' +
    '5. Click on Diet sticker option',
  expectedResults: 'Diet sticker PDF will open',
  automationManual: 'Automated',
  priority: 'P3',
  createdBy: 'Sumitha',
  createdAt: 45158,
  lastModifiedDate: 45261,
  linkedStories: 'HC-012, HC-054',
  risk: 'Low',
  version: 'v11.0',
  type: 'UI',
  createdAt: 2026-06-25T07:07:13.060Z,
  embeddingMetadata: {
    model: 'mistral-embed',
    cost: 0.000015316,
    tokens: 153,
    apiSource: 'mistral-batch',
    batchNumber: 103,
    createdAt: 2026-06-25T07:07:12.127Z
  }
}
```


},

id: undefined

}

Document {

pageContent: '',

metadata: {

_id: new ObjectId('6a3cd311783fc47dc0db2b55'),

module: 'Digital dietician',

id: 'TC_1047',

preRequisites: 'Dietician role, patient record in nutritional assessment dashboard and ward selected from setting',

title: 'Enable Special nutrition support',

description: 'Verify special nutrition support',

steps: '1. Login with credential\r\n' +

'2. Click on Dietician dashboard icon\r\n' +

"3. Click on 'NA status' tab\r\n" +

"4. Select the patient under 'not done' tab\r\n" +

'5. Click on special nutritional support slider',

expectedResults: 'Record will be generated in Special nutrition support dashboard ',

automationManual: 'Manual',

priority: 'P4',

createdBy: 'Sudarshan',

createdDate: 44597,

lastModifiedDate: 44664,

linkedStories: 'HC-002, HC-054',

risk: 'Low',

version: 'v13.0',

type: 'Functional',

createdAt: 2026-06-25T07:04:49.789Z,

embeddingMetadata: {

model: 'mistral-embed',

```
cost: 0.0000147100000000000001,  
tokens: 147,  
apiSource: 'mistral-batch',  
batchNumber: 1,  
createdAt: 2026-06-25T07:04:49.789Z  
}  
,  
id: undefined  
}  
Document {  
  pageContent: "",  
  metadata: {  
    _id: new ObjectId('6a3cd311783fc47dc0db2b56'),  
    module: 'Digital dietician',  
    id: 'TC_1048',  
    preRequisites: 'Dietician role, patient record in special nutritional support dashboard and ward selected from setting',  
    title: 'Complete Special nutritional support',  
    description: 'Verify completion of special nutritional support',  
    steps: '1. Login with credential\r\n' +  
      '2. Click on Special nutritional support icon\r\n' +  
      '3. Click on SNS status tab\r\n' +  
      '4. Select the patient from 'Not Done' tab\r\n' +  
      '5. Enter data on required field\r\n' +  
      '6. Click save\r\n' +  
      '7. Click Yes on pop up',  
    expectedResults: 'PDF will open with filled data.',  
    automationManual: 'Automated',  
    priority: 'P4',  
    createdBy: 'Sumitha',  
    createdDate: 45240,
```

```
lastModifiedDate: 45259,
linkedStories: 'HC-016',
risk: 'Low',
version: 'v2.0',
type: 'Functional',
createdAt: 2026-06-25T07:04:49.789Z,
embeddingMetadata: {
  model: 'mistral-embed',
  cost: 0.000014710000000000001,
  tokens: 147,
  apiSource: 'mistral-batch',
  batchNumber: 1,
  createdAt: 2026-06-25T07:04:49.789Z
},
id: undefined
}
Document {
  pageContent: "",
  metadata: {
    _id: new ObjectId('6a3cd35f783fc47dc0db361e'),
    module: 'Digital dietician',
    id: 'TC_1042',
    preRequisites: 'Dietician role, filled Nutritional assessment and ward selected from setting',
    title: 'Edit nutritional assessment',
    description: 'Verify edit functionality of nutritional careplan',
    steps: '1. Login with credential\r\n' +
      '2. Click on Dietician dashboard icon\r\n' +
      '3. Click on 'NA status' tab\r\n' +
      '4. Select the patient under 'Done' tab\r\n' +
      '5. Click Nutritional assessment tab\r\n'
```

```
'6. Click Edit on pop up\r\n' +
'7. Fill required data\r\n' +
'8. Click update \r\n' +
'9. Click Yes on pop up',
expectedResults: 'PDF will open updated record',
automationManual: 'Automated',
priority: 'P4',
createdBy: 'Sudarshan',
createdDate: 45210,
lastModifiedDate: 45251,
linkedStories: 'HC-002, HC-054',
risk: 'Medium',
version: 'v6.0',
type: 'Functional',
createdAt: 2026-06-25T07:06:07.717Z,
embeddingMetadata: {
  model: 'mistral-embed',
  cost: 0.000017282,
  tokens: 173,
  apiSource: 'mistral-batch',
  batchNumber: 56,
  createdAt: 2026-06-25T07:06:07.532Z
},
id: undefined
}
```

4. Chains: -

Create a file under src as chain.ts and add the below code

```
import { ChatGroq } from "@langchain/groq";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import "dotenv/config";

const prompt = ChatPromptTemplate.fromMessages([
  ["system", `A test case is considered Correct only if: It contains a
  Functionality section describing WHAT the system should do. It contains a
  Technical section describing HOW the functionality is implemented or
  validated. If both are present and valid, respond with exactly:

  Correct.

  Otherwise respond with exactly:

  Not correct.

  Do not provide any explanation.`],
  ["user", "{text}"]
]);

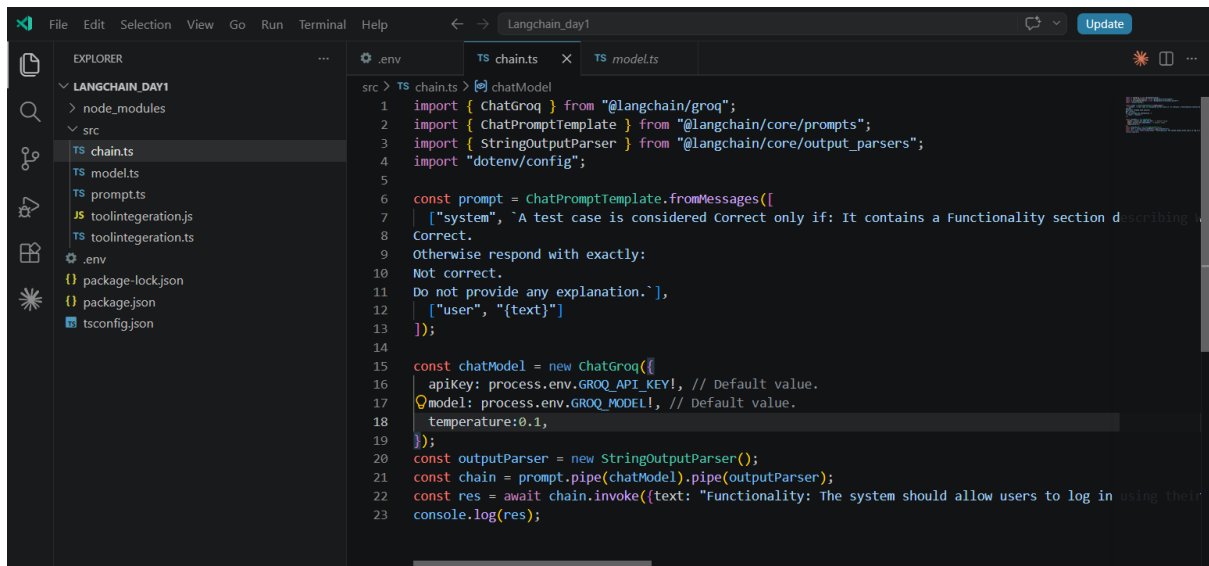
const chatModel = new ChatGroq({
  apiKey: process.env.GROQ_API_KEY!, // Default value.
  model: process.env.GROQ_MODEL!, // Default value.
  temperature: 0.1,
});

const outputParser = new StringOutputParser();

const chain = prompt.pipe(chatModel).pipe(outputParser);

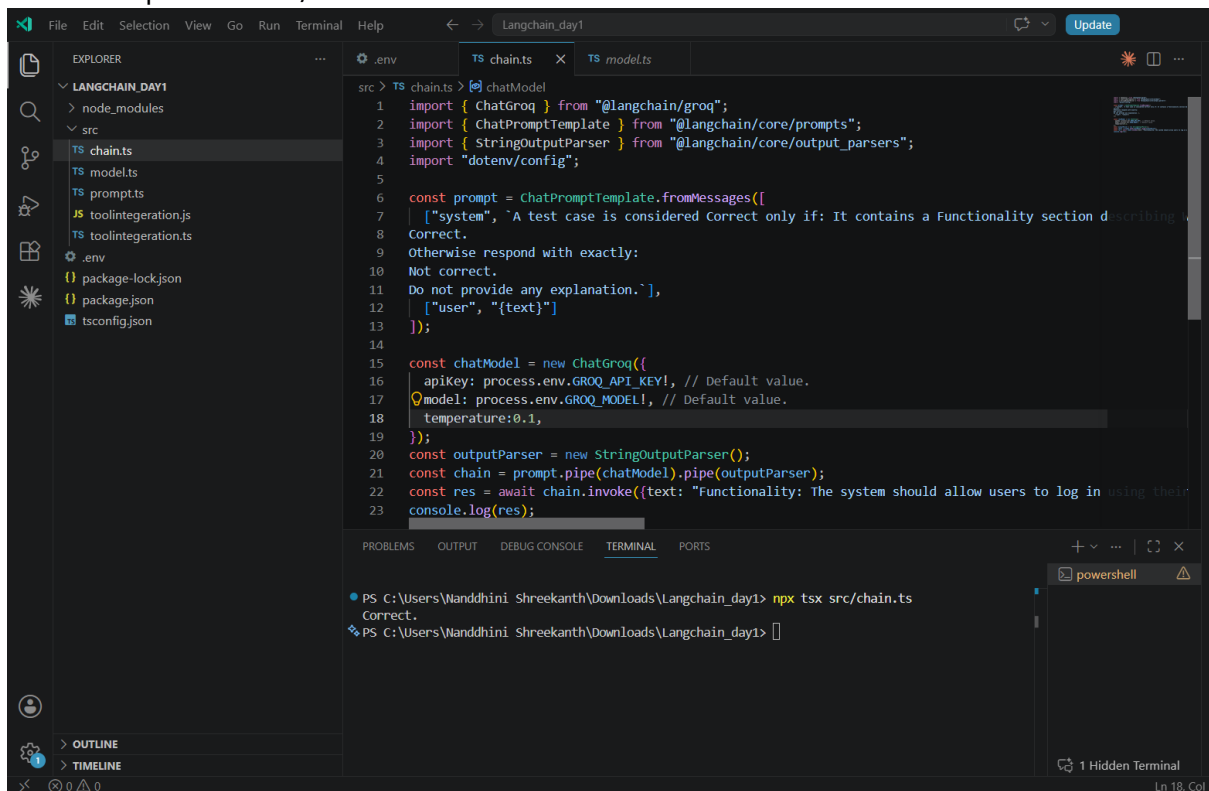
const res = await chain.invoke({text: "Functionality: The system should allow
users to log in using their email and password. \n\nTechnical: The login
process should validate the email format and check the password against the
stored hash."});

console.log(res);
```



```
src > TS chain.ts > chatModel
1 import { ChatGroq } from "@langchain/groq";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parsers";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7   ["system", "A test case is considered Correct only if: It contains a Functionality section describing v
8   Correct.
9   Otherwise respond with exactly:
10  Not correct.
11  Do not provide any explanation.`,
12   ["user", "{text}"]
13 ]);
14
15 const chatModel = new ChatGroq({
16   apiKey: process.env.GROQ_API_KEY!, // Default value.
17   model: process.env.GROQ_MODEL!, // Default value.
18   temperature: 0.1,
19 });
20 const outputParser = new StringOutputParser();
21 const chain = prompt.pipe(chatModel).pipe(outputParser);
22 const res = await chain.invoke({text: "Functionality: The system should allow users to log in using their
23 console.log(res);
```

Execute `npx tsx src/chain.ts`



```
src > TS chain.ts > chatModel
1 import { ChatGroq } from "@langchain/groq";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parsers";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7   ["system", "A test case is considered Correct only if: It contains a Functionality section describing v
8   Correct.
9   Otherwise respond with exactly:
10  Not correct.
11  Do not provide any explanation.`,
12   ["user", "{text}"]
13 ]);
14
15 const chatModel = new ChatGroq({
16   apiKey: process.env.GROQ_API_KEY!, // Default value.
17   model: process.env.GROQ_MODEL!, // Default value.
18   temperature: 0.1,
19 });
20 const outputParser = new StringOutputParser();
21 const chain = prompt.pipe(chatModel).pipe(outputParser);
22 const res = await chain.invoke({text: "Functionality: The system should allow users to log in using their
23 console.log(res);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Nandhini Shreekanth\Downloads\Langchain_day1> npx tsx src/chain.ts
Correct.
PS C:\Users\Nandhini Shreekanth\Downloads\Langchain_day1>
```